arXiv:2611.12304v1 [cs.DC] 9 Nov 2026

Evolving Infra: Agent-Adaptive LLM Serving at Scale

Hao Kang[1,*], **Weili Xu**[2,*], **Ziyang Li**[*], **Zheng Du**[1,*], **Hanchen Li**[3], **Tian Xia**[3], **Song Bian**[4], **Tushar Krishna**[1], **Huizi Mao**[5], **Ion Stoica**[3], and **Song Han**[6]

[1]Georgia Institute of Technology, [2]University of Illinois Urbana-Champaign
[3]University of California, Berkeley, [4]NVIDIA, [5]OpenAI, [6]Massachusetts Institute of Technology

[*]Equal contribution. Corresponding author: hao.kang.sys@gmail.com

Abstract

Agentic workloads are a major source of LLM inference demand. Existing prefill–decode (PD) disaggregated serving systems offer limited flexibility for runtime adaptation of compute allocation and KV cache management, and rely on coarse signals that only partly capture the diverse and dynamic agent workload. These limitations lead to load imbalance between prefill and decode nodes and eviction of KV caches that agents will reuse. We present EVOLVING INFRA, an agent-adaptive serving framework that adapts compute allocation to agent workloads and KV cache management to agent context management at runtime. We decouple compute allocation from key–value (KV) cache management, allowing nodes to switch efficiently between prefill and decode. Building on this design, an online planner matches the PD split to workload composition, while an agent-aware distributed KV manager guides cache eviction and transfer with signals from agent context management. Across coding, math, and computer-use tasks with GLM-5.2 and Kimi K2.6 on up to 256 GB300 GPUs, EVOLVING INFRA reduces mean end-to-end request latency and P95 TTFT by up to 4.1× and 14× over Dynamo, NVIDIA's production framework.

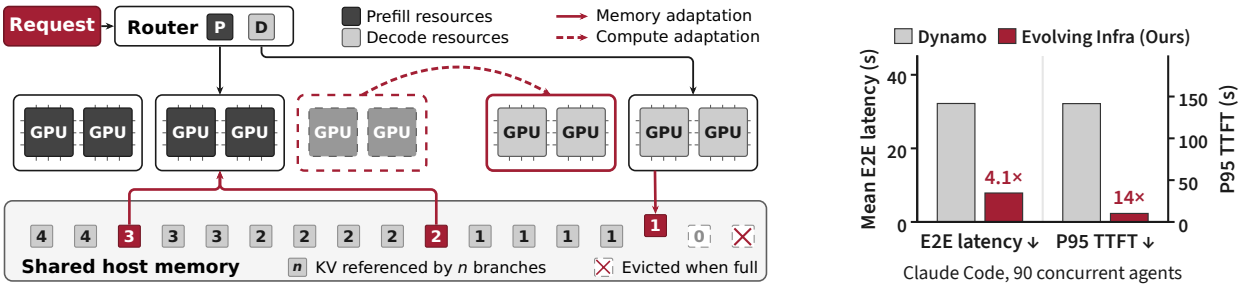
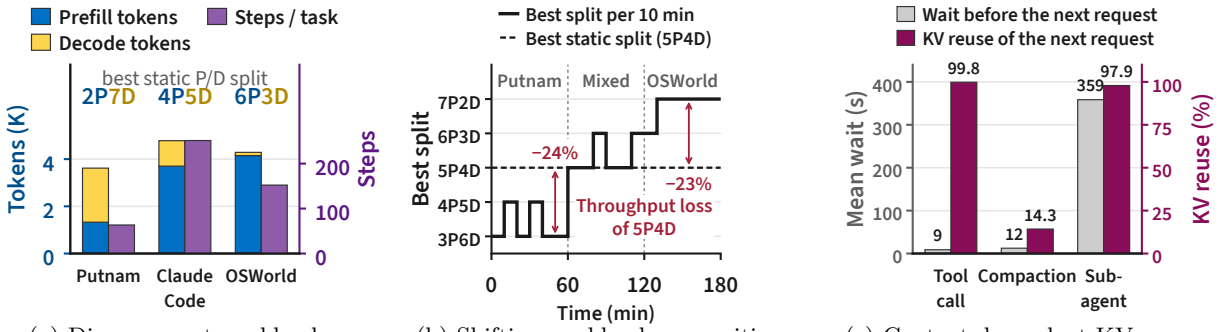


Figure 1: **Overview and speedup of EVOLVING INFRA.**

1 Introduction

Agentic workloads account for a large and growing share of LLM inference [9, 30], spanning software engineering [22], math research [38], and computer use [5]. Agents execute these tasks in steps, each consisting of a model request and the tool calls it triggers, with harnesses updating the context between steps. Successive requests therefore typically share long prefixes, creating opportunities for key–value (KV) cache reuse across consecutive steps.

These workloads place diverse demands on prefill and decode resources. Differences in tasks, harnesses, and models lead to different amounts of uncached input and generated output per request, making some agent workloads prefill-heavy and others decode-heavy (Figure 2a). The agent workload composition, i.e., the proportions of different agent workloads served by a cluster, also varies over time. A production study of 11.7 million requests finds that user-triggered task starts follow workday rhythms, whereas automatically triggered task starts are more evenly distributed throughout the day [79]. In prefill–decode (PD) disaggregation, the allocation of nodes (the PD split) that best balances prefill and decode can change as this composition shifts(Figure 2b).



(a) Diverse agent workloads (b) Shifting workload composition (c) Context-dependent KV reuse

Figure 2: **Agentic workloads vary across agents, over time, and within each agent.** (a) Putnam-Bench [Putnam; 62], Claude Code [CC; 51], and OSWorld 2.0 [OSWorld; 77] agents differ in prefill and decode work per request, so their best static splits range from 2P7D to 6P3D (xPyD: x prefill and y decode nodes). (b) As the workload composition shifts from Putnam to OSWorld, the best split per 10-minute window moves from 3P6D to 7P2D, and the best static split (5P4D) loses up to 24% of throughput in the Putnam and OSWorld periods. (c) In CC, 98% of the prompt after a 359 s sub-agent call matches earlier KV, but only 14% after a 12 s compaction, so recency does not predict KV reuse.

Meanwhile, within an agent, context management changes which prefixes subsequent requests can reuse. Harnesses compact histories, fork and create sub-agents [7, 47]. Each fork or sub-agent starts a new *branch*, a context that has its own sequence of requests and remains *active* until it finishes. A branch waiting for a tool or sub-agent may reuse its existing prefix after a long pause, whereas compaction can remove recently accessed history from the subsequent requests of a branch (Figure 2c). Access recency alone thus does not reveal which cached prefixes remain useful to active branches.

Serving agents efficiently thus requires adapting compute allocation to diverse agent workloads and their shifting composition, and KV cache management to agent context management. Production systems [40] serve agentic workloads with PD disaggregation [80] and manage compute and memory with three components. A planner determines the compute allocation between prefill and decode by assigning nodes to each phase [43, 69]. A cache-aware router routes each request to a node with matching prefix KV [45, 58]. A memory pool shares and evicts KV across nodes [50, 72]. However, agentic workloads expose three problems in existing systems.

Costly phase switches. Each adaptation of the compute allocation switches nodes between phases, and such switches are expensive in existing systems. Some [43] relaunch serving workers to change their phase, which takes minutes and removes the node from service during the switch. Others switch without a restart [69] but strand the node's cached KV, forcing later requests to recompute their prefixes. In our measurements on Claude Code traces, Dynamo takes 681 s per switch on average, while Arrow loses 55.5% of the available KV reuse over 34 in-place switches (Figure 3a).

Misestimated compute demand. A PD split profiled offline for each agent workload is costly to obtain and cannot follow the shifting workload composition. Even the best static split loses up to 24% of throughput in some periods relative to the best split of each 10-minute window (Figure 2b). Online planners also misestimate demand. They adjust the PD split using coarse signals such as queue depth [43] or predicted time to first token (TTFT) [31]. Context management, such as spawning several sub-agents at once, can create short bursts of prefill work that perturb these signals and trigger costly phase switches without sustained throughput gains. Some planners also estimate prefill work from the full input length [69], ignoring KV reuse and overestimating the number of prefill nodes needed (Figure 3b).

Misjudged KV reuse. Existing systems misjudge KV reuse when they route requests and evict KV. Requests of the same branch share long prefixes. Cache-aware routers [58] concentrate these requests on the few prefill nodes holding their prefixes, overloading them and raising P95 TTFT. Routers [45] that trade cache locality for load balance can route requests to nodes that must recompute their prefixes. The memory pool avoids this recomputation, but its KV fetches can delay prefill and leave duplicate copies in the pool (Figure 3c). The memory pool and node caches typically evict KV in least recently used (LRU) order [50, 78]. They thus drop the KV of branches waiting for tools, sub-agents, or users while keeping the history replaced by compaction, the contexts of finished sub-agents, and duplicate copies left by KV transfers.

Two limitations underlie these problems. First, compute allocation and KV cache management remain coupled. Engines index cached KV in local state. Switching a node from prefill to decode thus discards or strands its cached KV. Shared KV pools avoid this coupling, but keep the allocation fixed or change it by relaunching nodes. Second, the three components rely on coarse signals, such as request latency, input lengths, and access history, which do not fully reveal the agent workload or context management. To our knowledge, Evolving Infra is the first agent-adaptive serving framework, adapting compute allocation to diverse agent workloads and their shifting composition, and KV cache management to agent context management at runtime. We make three contributions.

Decoupled compute allocation and KV cache management. To reduce the cost of phase switches, Evolving Infra separates the phase of each node from the KV storage. Each node supports both phases, and the planner reassigns its phase at runtime without a restart. KV resides in a shared pool formed by the host memory of all nodes, and a cluster-wide KV directory records the number of active branches that reference each copy. Neither the pool nor the directory is tied to node phases, and a phase switch leaves the KV readable by all nodes. The directory updates these counts in response to agent context management and KV transfers between nodes (Section 4).

Agent-adaptive compute allocation. To precisely size the PD split for the agent workload composition, Evolving Infra learns iteration-time models of prefill and decode online, without offline profiling (Section 5.1). The prefill model incorporates KV reuse, enabling accurate estimation of prefill time on agentic workloads. The planner evaluates these models on the average work of a request over each planning window. These averages characterize the workload itself rather than the momentary load of the cluster. The planner switches only when the predicted throughput gain of a new split persists across the window, which keeps short bursts from triggering switches (Section 5.2).

Agent-adaptive KV cache management. To keep the reusable KV of active branches while balancing the load of prefill nodes, Evolving Infra routes requests and evicts KV according to agent context management. It fetches remote KV asynchronously while a request is queued, which allows the router to send each prefill request to the node with the shortest predicted TTFT. The directory shows that no active branch references replaced histories, the contexts of finished sub-agents, or duplicate copies left by transfers, and Evolving Infra evicts this KV first (Sections 4.2, 4.3, and 5.3).

We evaluate Evolving Infra on coding, computer-use and math agents, serving GLM-5.2 and Kimi K2.6 on 36 GB300 GPUs. When serving a single agent workload at a **stationary load**, Evolving Infra achieves up to 2.7× the throughput of Dynamo and up to 38% more than the best static split. Reduces the mean end-to-end (E2E) request latency by up to 4.1× and P95 TTFT by up to 14× relative to Dynamo (Figure 1). For a **shifting workload composition** from math to computer-use agents, Evolving Infra adapts the PD split online, achieving 2.1× the throughput of Dynamo and 28% more than the best static split (Section 6.3).

2 Background

2.1 Diverse and dynamic agentic workloads

Workload composition. An agent solves a task in steps orchestrated by a harness. Each LLM request appends new tokens from tools or the harness to a prefix whose KV was computed in earlier steps. The inference engine fetches the cached prefix KV, prefills the uncached tokens, and decodes the output tokens. The amounts of prefill and decode work differ across tasks, harnesses, and models. For example, the workload of a computer-use agent on OSWorld is prefill-heavy because each step appends a new screenshot to the context, whereas the workload of a math agent on Putnam is decode-heavy because each request generates a long reasoning trace (Figure 2a). The best PD split thus differs across agent workloads, and finding it requires costly offline profiling (Section 6.4). The workload composition of a cluster also changes over time. User-triggered agent tasks starts follow workday rhythms, whereas automatically triggered tasks starts are more evenly distributed throughout the day [79]. In production LLM serving, the mean input length of requests to one model rises by 13% from midnight to afternoon, while the mean output length falls by 18% [70]. The best PD split thus shifts over time (Figure 2b). The PD split must therefore change over time to avoid load imbalance between prefill and decode nodes.

Context management. The harness typically extends the context of a branch with model outputs and tool results, and consecutive requests therefore share prefixes. Harnesses also manage context through forking, sub-agent spawning, and compaction. *Forking* creates new branches that reuse the current context, while spawning a *sub-agent* starts a separate branch with a new context. *Compaction* replaces the history of a branch with a summary [6, 47]. Once compaction completes, even recently accessed KV of the replaced history is no longer needed. In contrast, KV can remain idle during a long wait for tools or sub-agents, yet still be reused when the branch resumes. In GitHub Copilot, the median gap between consecutive turns is 172 s [30]. Whether KV will be reused thus depends on whether an active branch still includes it in its context, not merely on access recency. Together with the shift in workload composition, these dynamics challenge the planner, the router, and the memory pool of existing serving systems, which the rest of this section reviews.

2.2 Existing PD disaggregated serving systems

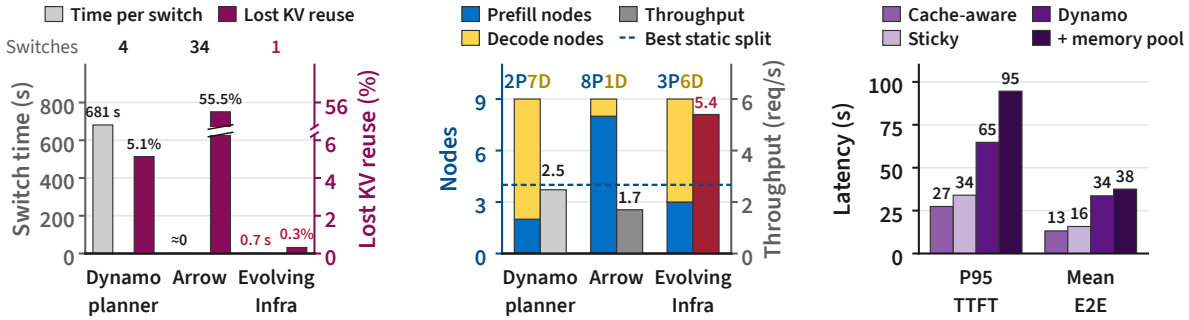
Engines. By default, vLLM and SGLang fix the phase of each engine at launch in PD disaggregation mode [53, 64]. For example, each SGLang engine builds only the scheduling loop and KV transfer path of its own phase. A prefill engine queues requests and sends their KV to decode engines, whereas a decode engine reserves KV slots, receives KV, and captures CUDA graphs for decoding. Each engine also indexes cached KV in local state, such as the prefix tree of a prefill engine. A phase switch thus requires relaunching the engine with a new configuration and discards its cached KV.

Planners. A planner sets the PD split by switching nodes between prefill and decode as compute demand changes. Phase switches in existing planners are costly. A switch in Dynamo relaunches the engine and keeps the node out of service for minutes [43]. A switch in Arrow [69] and xLLM [31] reassigns the node to the other phase without a relaunch but strands its cached KV. The split also follows coarse signals, namely prefill queue depth and decode KV utilization in Dynamo [43], and TTFT and per-token latency targets in Arrow, xLLM, and Llumnix [32]. These planners predict TTFT from the full input length [31, 33, 69], which counts cached prefixes as prefill work and overestimates prefill demand. Sub-agents spawned at once by the harness create a short burst of requests. The burst raises queueing delay and TTFT but leaves the long-term balance between prefill and decode workloads unchanged. These signals thus misestimate demand and trigger frequent phase switches, each costing minutes of downtime or the cached KV of the switched node (Section 3).

Routers. A router assigns each request to a prefill and decode node. For prefill, the SGLang Model Gateway and the Dynamo router weigh prefix-cache hits against node load [45, 58]. The Dynamo router keeps a cluster-wide prefix tree that records which KV blocks each node holds, and the engines update the tree through KV events [45]. For each request, the router uses this tree to count the prefix blocks that each node already holds. These routers choose where a request runs but do not move its KV. Without a memory pool, a request thus either queues at a node that holds its prefix, raising TTFT, or recomputes the prefix on another node, lowering throughput.

Memory pools. A memory pool extends the capacity for reusable KV beyond GPU memory. HiCache caches KV in GPU memory (L1) and host memory (L2) on each node and shares KV across nodes through an optional storage tier (L3), such as Mooncake Store [50, 72]. Shared KV moves from the L2 of the source node through L3 to the L2 of the destination node, leaving copies in L3 and in the L2 of both nodes. HiCache loads the fetched KV into GPU memory only when the request starts its prefill [52]. This load therefore delays prefill and raises TTFT (Section 3.3). These pools index KV by content and typically evict KV in LRU order. LRU eviction thus cannot distinguish the KV of a branch that waits for a tool from the history that compaction has replaced. Agent-aware systems such as Continuum and ThunderAgent retain the KV of agents that wait for tools [23, 27], but assume that each context only grows and do not track the duplicate copies that KV transfers leave across nodes.

In summary, these systems couple compute allocation with KV cache management, making phase switches costly. Their planners rely on coarse signals that do not fully reveal the workload composition, and their routers trade cache locality against load balance. Their memory pools load fetched KV only when prefill starts, evict KV by access recency, and leave duplicate copies after KV transfers. Agent-aware systems retain the KV of waiting agents but assume that contexts only grow. Section 3 quantifies the resulting losses.



(a) Costly phase switches (b) Misestimated compute demand (c) Misjudged KV reuse

Figure 3: **Costly phase switches, misestimated compute demand and misjudged KV reuse in PD disaggregation systems.** Lost KV reuse is $1 - h/h^*$, with h the KV hit rate and h^* that with an unlimited KV cache shared by all nodes and no phase switches. (a) Switch time and lost KV reuse; once its running requests finish, a EVOLVING INFRA node switches in 0.7 s, and EVOLVING INFRA loses 0.3% of KV reuse. (b) Final split and throughput; EVOLVING INFRA achieves over twice the throughput of either planner. (c) P95 TTFT and mean E2E latency at the best static split, 4P5D; + memory pool adds shared host memory to Dynamo.

3 Motivation

This section profiles PD-disaggregated serving systems on agentic workloads and quantifies three sources of inefficiency, namely costly phase switches, misestimated compute demand, and misjudged KV reuse. We use the testbed of Section 6.1, nine nodes of 4×GB300 GPUs that serve GLM-5.2 with TP4, and replay the Claude Code traces with 90 concurrent agents for one hour after a 30-minute warmup. Dynamo runs its planner at the default throughput target with the SGLang Model Gateway. Arrow's code is not public, so we reimplement its planner and scheduler on the symmetric engines of EVOLVING INFRA (Section 4.1). Both planners start from 6P3D, and each node runs HiCache with LRU eviction. For routing (Section 3.3), we run SGLang in its PD disaggregation mode at the best static split, 4P5D, and compare the cache-aware and sticky-session policies of the SGLang Model Gateway, the Dynamo KV router [45], and a variant of the Dynamo KV router whose nodes fetch prefixes from a shared host-memory pool (+ memory pool).

3.1 Costly phase switches

Every phase switch takes time, loses KV reuse, or both (Figure 3a), because engines fix their phase at launch and index cached KV locally (Section 2.2). The Dynamo planner relaunches the worker of a node at every switch. A switch takes **681** s on average, during which the node serves no requests. Arrow switches nodes in place without this delay, but the KV of a switched node becomes unreachable for prefill. Arrow misses 55.5% of the cache hits that it would achieve with an unlimited KV cache shared by all nodes and no phase switches, and Dynamo misses 5.1%. We call this fraction of missed hits the *lost KV reuse*. Avoiding both costs requires an engine that serves both phases without a relaunch and a KV pool that every node can read in either phase.

3.2 Misestimated compute demand

Both online planners size the split from coarse signals (Section 2). The Dynamo planner and Arrow thus deliver 49% and 65% less throughput than the best static split(4.9 req/s), respectively (Figure 3b). These signals misjudge the work of agents in opposite directions. Arrow predicts prefill time from the full input length, which counts cached prefixes as work, and it also reacts to short prefill bursts, such as several sub-agents that start together. Arrow thus drifts from 6P3D to 8P1D, and switches 34 times during the run, each time incurring the costs in Section 3.1. The Dynamo planner instead follows queue depth and KV utilization. Long agent contexts keep the KV utilization of decode nodes high, and the planner drifts past 4P5D to 2P7D.

Sizing the split well requires a model of the average prefill and decode work that a request needs after KV reuse. Short bursts of sub-agents and forks raise the momentary load but barely change the average.

3.3 Misjudged KV reuse

Routers misjudge the KV reuse of agents in either direction (Figure 3c). The cache-aware and sticky policies of the SGLang Model Gateway keep an agent and its forked branches on the few prefill nodes that hold their prefixes. The P95 TTFT thus reaches 27 s and 34 s. The load-aware Dynamo KV router [45] sends more requests to nodes that lack their prefixes, which recompute them, and the P95 TTFT reaches 65 s. With the shared memory pool, these nodes fetch the prefixes instead, yet the P95 TTFT rises to 95 s. Because the pool loads the fetched KV into GPU memory only when the request begins its prefill, the load adds to the prefill time instead of overlapping with queueing (Appendix D.6). The duplicate copies that fetches leave also occupy host memory and displace the KV that agents still need.

Eviction misjudges KV reuse as well. HiCache evicts the least recently used KV, whereas the reuse of agent KV depends on whether an active branch still includes it (Section 2). HiCache thus keeps histories that compaction replaced but evicts the KV of branches that wait for tools, sub-agents, or users. On the CC traces, these evictions cause 39% of the KV reuse that Dynamo loses (Appendix C.1). Reusing KV well requires a memory pool that hides fetch delays, routing that balances load, and eviction that follows agent context management and removes duplicate copies.

4 Decoupled and Agent-Aware Infrastructure

This section presents the infrastructure that enables efficient online adaptation (Figure 1). The symmetric engine (Section 4.1) allows nodes to switch phases without a relaunch, and the shared KV pool (Section 4.2) keeps KV readable by all nodes in either phase. Together, they meet the two needs in Section 3.1 and decouple compute allocation and KV cache management. The KV directory (Section 4.3) counts the active branches that reference each KV block on each node and updates these counts as branches fork, compact, or finish and as KV moves between nodes. These counts identify the KV that no active branch reuses (Section 3.3).

4.1 Symmetric engines for online phase switches

To reduce the cost of online prefill–decode phase switches (Section 3.1), we build a symmetric engine that supports both phases, so that every node can serve either phase. With this engine, the router separates prefill and decode at runtime rather than through a static deployment configuration. Unlike SGLang (Section 2), whose prefill and decode engines run different scheduling loops and fix their phase at launch by default, our engine extends SGLang with one scheduling loop that serves both phases and a KV transfer protocol that moves KV between any two nodes in either direction. The protocol reuses the RDMA transport of Mooncake [50] and keeps working after a node switches its phase. Every node of Evolving Infra runs this engine, and a single router routes prefill requests to one set of nodes and decode requests to the remaining nodes (Figure 1). When the planner reassigns the phase of a node (Section 5.2), the router changes the type of requests that it routes to the node, and the engine serves the new phase without a restart.

4.2 Shared host-memory KV pool

To keep KV readable by every node in either phase (Section 3.1), Evolving Infra pools the host memory (L2) of all nodes into a shared KV pool. Following HiCache [72], each node caches KV in GPU memory (L1) and writes it through to its L2 in both phases. A prefill node locates reusable KV through the directory (Section 4.3) and pulls it from the L2 of a peer over RDMA. Unlike the storage tier of SGLang, which stages shared KV in a separate store, the pool reads directly from the L2 of a peer and avoids this extra copy (Appendix A.2). Deduplication across the GPUs of a node further increases its effective capacity (Appendix A.5). Decode nodes only write to the pool, since a fetch from host memory would delay decoding (Appendix D.6). They receive the KV of each request from the prefill node through the faster GPU-to-GPU transfer, and the KV that they generate stays in their L2 for the next request of the branch.

A fetch from a peer, however, delays the prefill of its request (Section 3.3). HiCache prefetches KV only into host memory while a request waits and loads it into GPU memory layer by layer during prefill. Evolving Infra instead completes the whole fetch while the request waits in the queue. The node pulls the KV from the L2 of the peer into its own L2, loads it into GPU memory, and broadcasts it to all GPUs of the node before the prefill begins.

When the planner selects a new split (Section 5.2), the router moves the queued requests of each switching node to other nodes of its current phase. The running requests of the node finish in place, after which the node switches its phase. The KV of the switching node stays in the pool and remains readable by all nodes throughout the switch (Section 3.1).

4.3 Agent-aware cluster-wide KV directory

To identify the duplicate copies that KV transfers leave and the histories that context management drops (Section 3.3), the router (Figure 1) maintains a cluster-wide KV directory. Following the Dynamo KV indexer [45], the directory is a prefix tree that records KV placement in GPU and host memory of each node. The router updates this tree with the KV events that the engines publish (Section 2). The directory also records, for each block on each node, how many active branches reference it (Figure 4).

The router updates these counts from two identifiers. The harness tags each request with the ID of the agent branch that issues it and reports when a branch finishes. The router maintains a version for each branch and increments it when the context of the branch is rewritten. The engine on each node then updates the eviction priority of the KV blocks in its host memory according to the counts that the router sends (Section 5.3).

Both identifiers follow agent context management. When the harness forks a context or spawns a sub-agent, it sends the ID of the new branch to the router. The router then adds one count to each block that the new branch reuses on the node that serves it. For consecutive requests of the same branch, the router compares the new request with the previous request of the branch. If the new request only appends tokens, the router adds counts only to the newly appended blocks on the routed node (Figure 4a). The reused blocks keep their counts, since the branch was already counted on them in earlier steps. If the context is rewritten by compaction, the router increments the version. It then releases the counts on the blocks that the new context drops and adds counts to the blocks that it newly includes (Figure 4b). When the harness reports that a branch has finished, the router releases all counts of the branch.

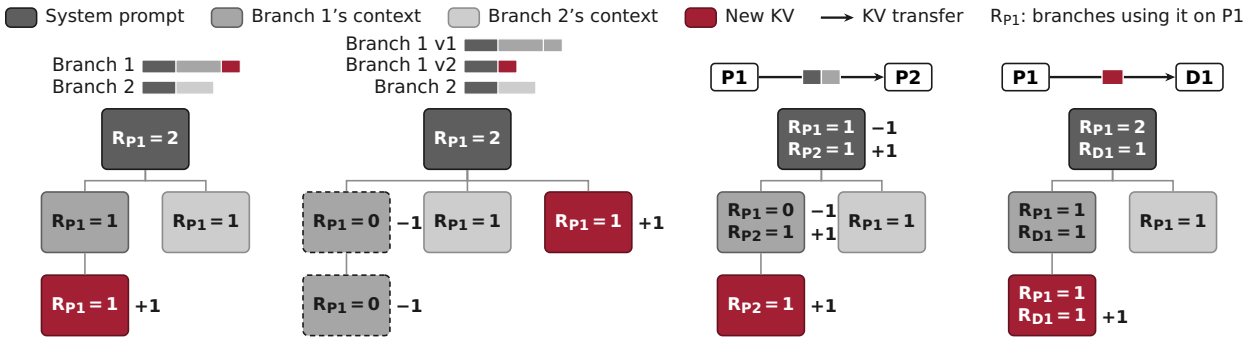
Counts also follow KV transfers. A transfer between two nodes of the same phase (P2P or D2D) moves the counts of the branch, since the branch reuses only the target copy in that phase. The source releases them once the target holds the current version (Figure 4c). A transfer across phases (P2D or D2P) keeps the counts on both nodes, since the next step of the branch needs the copies in both phases (Figure 4d). A node that switches from decode to prefill resets its counts to free its memory, whereas a node that switches to decode keeps them to preserve its KV. Appendix E.1 and Appendix E.2 give these rules in pseudocode. As a result, each active branch holds counts on at most one copy per phase.

5 Agent-Adaptive Compute Allocation and KV Cache Management

Based on the infrastructure in Section 4, this section presents how Evolving Infra adapts compute allocation and KV cache management to agents at runtime. We first characterize the work of each request and learn iteration-time models of each node online (Section 5.1). Section 5.2 then presents the planner, which uses these models to size the PD split for diverse agent workloads and their shifting composition. Given the split, Section 5.3 presents KV-reuse-aware eviction and routing, which preserve the KV that active branches still reuse and balance prefill work across nodes. Table 1 summarizes the notation.

5.1 Prefill and decode iteration-time models

As Section 3.2 shows, sizing the split requires a model of the average prefill and decode work that a request needs after KV reuse. This section models how long a request with given work occupies a node, and Section 5.2 evaluates this model on the average work over each planning window. A node runs its requests in iterations,



(a) Consecutive requests (b) Compaction (c) P2P and D2D (d) P2D or D2P

Figure 4: **Counts follow agent context management and KV transfers.** Each block of the cluster-wide prefix tree shows, for each node that holds it, the number of active branches that reference it there: R_{P1} and R_{P2} on prefill nodes P1 and P2, and R_{D1} on decode node D1. Tags +1 and −1 mark the counts that the step changes, and both branches start on P1. (a) A consecutive request counts only its new block, and reused blocks keep their counts. (b) Compaction increments the version of branch 1 from v1 to v2, so its old blocks drop to $R_{P1} = 0$ (dashed). (c) When branch 1 moves from P1 to P2, P2 counts its KV and P1 releases its counts. (d) When branch 1 alternates between P1 and D1, the target nodes count its KV, and each step sends only new KV.

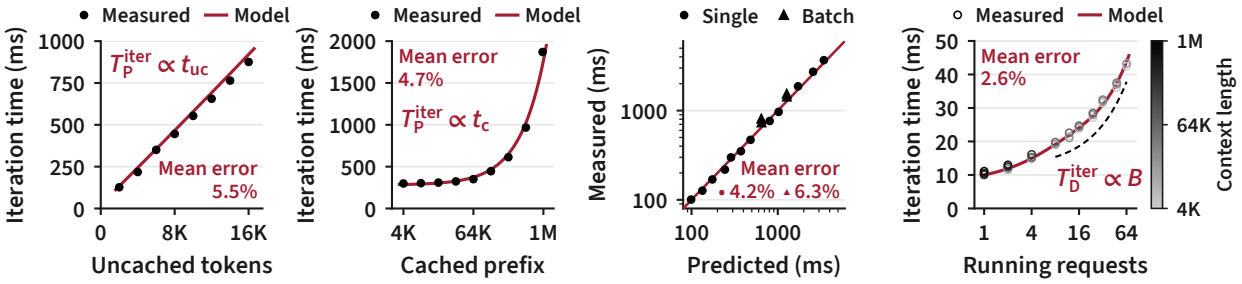
Table 1: Summary of notation for requests and the cluster.

Notation	Description
t_{uc}, t_c	Uncached and cached prefill tokens of a request
t_{out}, t_{acc}	Output tokens; mean accepted tokens per decode iteration
t_{fwd}	Tokens that each running request feeds into a decode iteration
w	Work of a request, defined by $(t_{uc}, t_c, t_{out}, t_{acc})$
N, n	Number of nodes; index of a node
$\mathcal{P}$, $\mathcal{D}$	Sets of prefill and decode nodes
B	Number of running requests on a decode node

each of which is a forward pass of the model. Under continuous batching [76], each iteration serves a batch of requests, and requests join or leave the batch between iterations. Accurate predictions of iteration time thus allow Evolving Infra to estimate the prefill and decode throughput of each node and the waiting time of prefill requests. For each phase, we model the iteration time from the requests in its batch and derive the number of requests that each node completes per second. We describe each request in a batch by its work $w = (t_{uc}, t_c, t_{out}, t_{acc})$, where Table 1 defines each component. The router obtains t_c from the longest cached prefix that the KV directory finds for the request (Section 4.3), and t_{uc} counts the remaining input tokens. Separating the two token lengths allows the models to predict prefill time accurately, even when agents reuse most of their context.

The models rest on two premises. First, the kernels of a forward pass run one after another, and the iteration time is the sum of their execution times. Second, as in the roofline model [68], each kernel is limited either by the compute throughput or by the memory bandwidth of the GPU. A compute-bound kernel runs in time proportional to its arithmetic operations, whereas a memory-bound kernel runs in time proportional to the bytes that it reads and writes.

Prefill iteration-time model. Prefill spends most of its time in compute-bound kernels [80]. Under chunked prefill [1], each prefill iteration processes the uncached tokens of several requests up to a fixed token budget. The execution time of the linear layers grows in proportion to the uncahced tokens of the batch, and that of the attention grows in proportion to its query–key pairs. Under causal attention, the i-th uncached token attends to the t_c cached tokens and to the first i uncached tokens, including itself. A request therefore computes $A = t_{uc}t_c + t_{uc}(t_{uc}+1)/2$ query–key pairs (Figure 5b). A request reuses its cached prefix from GPU memory or from host memory. Asynchronous fetches hide most of the restore from host memory (Section 4.2),



(a) Prefill iteration time vs. uncached tokens

(b) Prefill iteration time vs. cached prefix

(c) Predicted vs. measured iteration time

(d) Decode iteration time vs. running requests

Figure 5: **The iteration-time models closely track the measured iteration times.** Points are GPU times per iteration of GLM-5.2 on four GB300 GPUs, and red lines are the models in Equations (1) and (2). We fit the prefill model on 20 single-request settings and the decode model on the ten points with a 16K context, and each panel reports the mean error on the remaining points. (a) The prefill iteration time grows nearly linearly with the uncached prefill tokens at a 128K cached prefix. (b) It grows linearly with the cached prefix at 8K uncached prefill tokens (log-scale axis). (c) Outside the fitted settings, the mean error is 4.2% for single requests and 6.3% for batches of 2–8 requests. The panel shows 16 of the 60 measured prefill iterations, evenly spaced and including the four batches with the largest errors. (d) The decode iteration time follows the model as the running requests grow, and with 8 or more running requests, the context length (darker for longer) changes it by only 2–7%.

but each restored token still adds a small cost to the iteration, such as the remaining host-to-GPU copy. This cost grows in proportion to the restored tokens. Each iteration also pays a fixed cost, such as the time to launch its kernels. A prefill iteration on node n thus takes

$$T^{\text{iter}}_{\text{P},n} = c_{n,0} + c_{n,1} \sum t_{\text{uc}} + c_{n,2} \sum A + c_{n,3} \sum t_{\text{rst}}, \tag{1}$$

where the sums run over the requests of the batch, and t_{rst} is the number of tokens that a request restores from host memory. The coefficients $c_{n,0}$ to $c_{n,3}$ give the fixed cost of an iteration and the time per uncached token, per query–key pair, and per restored token. They depend on the model, hardware, and parallelism of node n, and Evolving Infra fits them **online**, as described at the end of this section. Appendix B.2 shows that Equation (1) also holds for sliding-window, linear, and sparse attention, including the DeepSeek Sparse Attention (DSA) of GLM-5.2.

Decode iteration-time model. Decode spends most of its time in memory-bound kernels [80], whose execution time grows in proportion to the bytes they read. Most of these bytes belong to the model weights. Each iteration reads the dense weights in full, which forms a fixed cost. In the mixture-of-experts (MoE) layers, the iteration reads only the weights of the routed experts that its tokens select. A decode iteration serves the B running requests of a node, and each request feeds t_{fwd} tokens into the iteration, a number that exceeds one under speculative decoding. With each token selecting k of the E routed experts, the $t_{\text{fwd}}B$ tokens select an expected fraction $\phi(B) = 1 - (1 - k/E)^{t_{\text{fwd}}B}$ of them (Appendix B.3). A decode iteration holds only a few tokens, and this fraction therefore changes with B. Each running request also adds about the same time to the iteration, for example to sample its next tokens and to draft tokens for speculative decoding. This part of the iteration time grows in proportion to B. The time to read the KV of running requests also grows in proportion to their total context length. GLM-5.2 and Kimi K2.6, however, stores little KV per token and reads only part of it under sparse attention (Appendix B.4). At 8 or more running requests, the context length thus changes the iteration time by only 2–7% (Figure 5d), and the model omits this term. A decode iteration on node n thus takes

$$T^{\text{iter}}_{\text{D},n}(B) = d_{n,0} + d_{n,1}\,\phi(B) + d_{n,2}\,B. \tag{2}$$

The coefficients $d_{n,0}$ to $d_{n,2}$ give the fixed cost of an iteration, the time to read the weights of all routed experts, and the time per running request. Like the prefill coefficients, they depend on the model, hardware, and parallelism of node n, and Evolving Infra fits them **online**.

Request time and node throughput. In Equation (1), every term except the fixed cost sums over the requests of the batch. A request thus occupies the prefill node for its own share of these terms plus part of the fixed cost, which we divide among the requests in proportion to their uncached tokens. With $\bar{t}_n$ the average number of new tokens per prefill iteration on node n and f_n the fraction of cached tokens that the node restores from host memory, a request with work w occupies node n for

$$T^{\text{req}}_{\text{P},n}(w) = c_{n,0}\, t_{\text{uc}}/\bar{t}_n + c_{n,1}\, t_{\text{uc}} + c_{n,2}\, A + c_{n,3}\, f_n\, t_{\text{c}} \tag{3}$$

in prefill. In decode, each iteration advances every running request by t_{acc} tokens on average, and a request needs $t_{\text{out}}/t_{\text{acc}}$ iterations to finish. Since the B running requests share each iteration, a request occupies node n for

$$T^{\text{req}}_{\text{D},n}(w) = T^{\text{iter}}_{\text{D},n}(B)\, t_{\text{out}}/\big(B\, t_{\text{acc}}\big) \tag{4}$$

in decode. Node n thus completes $\mu_{\text{P},n} = 1/T^{\text{req}}_{\text{P},n}(w)$ requests per second in prefill and $\mu_{\text{D},n} = 1/T^{\text{req}}_{\text{D},n}(w)$ requests per second in decode.

Online fitting. Evolving Infra fits both models purely online at the start of serving. During a warmup period, every node serves both phases in a collocated mode, and Evolving Infra records the prefill and decode work that each node completes in fixed windows. It fits the coefficients of both models for each node by nonnegative least squares, freezes them, and then starts PD disaggregation from the initial split. After warmup, Evolving Infra periodically remeasures only $\bar{t}_n$ and f_n, which change with the workload composition (Figure 2b). On controlled measurements on GB300, the model form predicts iteration time with a mean error below 7% outside the fitted settings (Figure 5).

5.2 Online phase-split adaptation

To size the PD split for diverse agent workloads and their shifting composition (Figure 2a and Figure 2b), the planner reevaluates the possible splits by their predicted throughput at the end of each planning window of length T_{win}. Each request passes through both phases, and the slower phase thus determines the cluster throughput,

$$\Theta(\mathcal{P}, \mathcal{D}; \bar{w}, \bar{q}) = \min\Big(\sum_{n\in\mathcal{P}} \mu_{\text{P},n},\ \sum_{n\in\mathcal{D}} \mu_{\text{D},n}\Big), \tag{5}$$

where $\mathcal{P}$ and $\mathcal{D}$ together contain the N nodes of the cluster. Here $\bar{w}$ is the average work of a request over the window, and $\bar{q}$ is the average number of requests in decode. Unlike queue depth, $\bar{w}$ characterizes the agent workload itself and barely changes during short bursts of sub-agents and forks. The planner evaluates $\mu_{\text{P},n}$ with $\bar{w}$ in Equation (3) and $\mu_{\text{D},n}$ with the batch $B = \bar{q}/|\mathcal{D}|$ in $T^{\text{req}}_{\text{D},n}$.

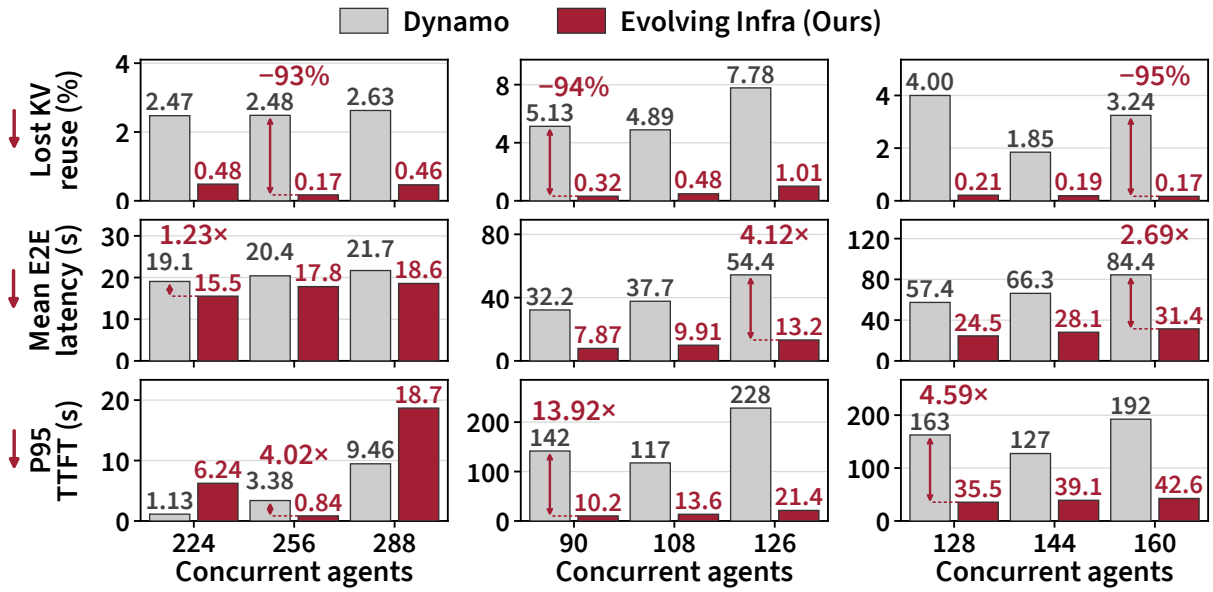
Since each switch has a cost (Section 3.1), the planner switches only when another split is statistically better than the current one. For each prefill count other than the current one, the planner forms a candidate split $(\mathcal{P}', \mathcal{D}')$. It divides the window into m slices and measures $\bar{w}_i$ and $\bar{q}_i$ on each slice i. The gain of the candidate on slice i is

$$g_i = \Theta(\mathcal{P}', \mathcal{D}'; \bar{w}_i, \bar{q}_i) - \Theta(\mathcal{P}, \mathcal{D}; \bar{w}_i, \bar{q}_i). \tag{6}$$

The candidate qualifies if its mean gain exceeds twice its standard error, $\bar{g} > 2\sigma_g/\sqrt{m}$, where $\bar{g}$ and σ_g are the mean and standard deviation of the gains g_i. This threshold keeps short bursts from triggering switches (Section 3.2). If any candidate qualifies, the planner switches to the one with the largest $\bar{g}$. It applies the new split through the phase switch in Section 4.1, and Appendix E.3 summarizes the planner.

5.3 KV-reuse-aware eviction and routing

Eviction. The rules in Section 4.3 keep the count of a copy positive only while an active branch may still reuse it on that node. A block with a zero count, such as the history that compaction replaced, the context of a finished sub-agent, or the source copy left by a same-phase transfer (Figure 4), is thus KV that no active branch will reuse. Each node therefore evicts the blocks in its host memory in ascending order of the key $(\mathbb{1}[R_b > 0], T_b)$, where R_b is the count of block b on the node and T_b is its last access time. The node thus evicts zero-count blocks first, in LRU order, and evicts blocks with positive counts only after no zero-count block remains. Unlike LRU (Section 3.3), this order evicts replaced histories before the KV of branches that wait for tools, sub-agents, or users.



(a) Putnam (GLM-5.2) (b) CC (GLM-5.2) (c) OSWorld (Kimi K2.6)

Figure 6: **Evolving Infra loses less KV reuse and achieves lower mean E2E latency than Dynamo in every setting.** Lost KV reuse equals $1 - h/h^*$, with h the KV hit rate and h^* the hit rate under unlimited KV cache capacity shared by all nodes and no phase switches. In each panel, an arrow marks the largest gap, and the number above it gives the gap as a relative reduction for lost KV reuse and as a ratio for latency.

Routing. With asynchronous fetches (Section 4.2) and the eviction policy above, KV transfers are fast and memory-efficient. The router therefore balances the predicted TTFT across prefill nodes. For a request with work w and prefill node n, the router predicts

$$\hat{T}_n(w) = T_n^{\text{rem}} + \textstyle\sum_{j \in Q_n} T_{\text{P},n}^{\text{req}}(w_j) + T_{\text{P},n}^{\text{req}}(w), \tag{7}$$

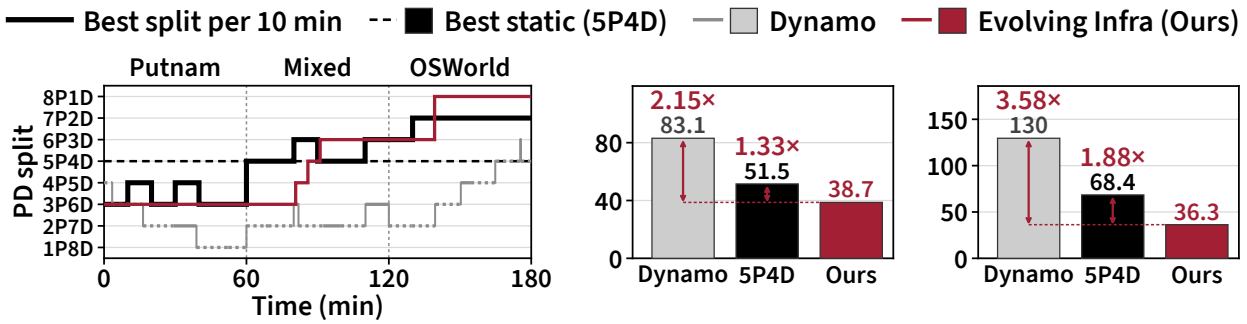
where T_n^{rem} is the remaining time of the running batch on node n, Q_n is the set of its queued requests, and each service time follows Equation (3). Appendix E.3 details how the prediction also charges the part of a fetch that outlasts the wait and the duplicate copy that the fetch creates. The router routes the request to the node with the smallest $\hat{T}_n(w)$, and that node fetches the KV of the request from the shared pool while the request waits. Unlike cache-aware routing (Section 3.3), this choice weighs the queue of each node against the KV that the node must fetch and avoids overloading nodes with popular prefixes.

6 Experiments

In this section, we evaluate Evolving Infra on coding, computer-use, and math agents served on 36 GB300 GPUs. Section 6.1 summarizes the setup. Section 6.2 evaluates each agent workload in isolation under a stationary load. Section 6.3 evaluates a shifting workload composition. Section 6.4 measures the sensitivity of Evolving Infra to the initial PD split, the contribution of each component, and the cost of online phase-split adaptation. Appendix D further evaluates Evolving Infra on up to 256 GPUs, in simulation at 3,072 GPUs, under open-loop arrivals, and with different parallelism for prefill and decode.

6.1 Experimental setup

Workloads. We deploy mini-SWE-agent [74] on PutnamBench [Putnam; 62] and Anthropic's computer-use agent [5] on OSWorld 2.0 [OSWorld; 77]. Putnam is a decode-heavy workload that runs a single agent branch per problem without context management. OSWorld is a prefill-heavy, multimodal, multi-agent workload that appends a screenshot to the context at every step (Figure 2a). The Claude Code traces [CC; 51], collected



(a) PD split over time (b) Mean E2E latency (s) (c) P95 TTFT (s)

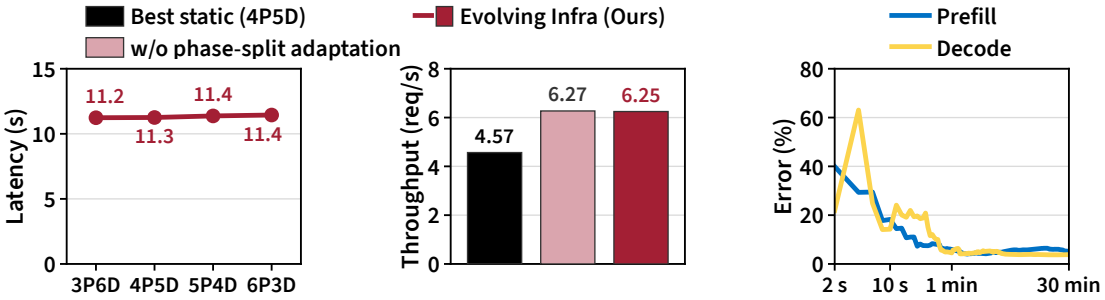
Figure 7: **Evolving Infra adapts the PD split as the workload composition shifts and achieves lower latency than Dynamo and the best static split.** (a) The best split per 10-minute window and the best static split are profiled under SGLang PD disaggregation, as in Figure 2b. The Dynamo line is dotted while one of its nodes relaunches and only eight nodes serve requests. (b, c) Over the whole run, the red number above a baseline bar gives its ratio to Evolving Infra.

from real users, form the most dynamic of the three workloads. They interleave sub-agents, compactions, and long pauses between requests (Figure 2c). Putnam and OSWorld thus differ most in their demand for prefill and decode, whereas CC exercises agent context management the most. We replay CC with the Trie replayer [8], which preserves the pauses between requests and the prefix sharing of the original traces. Trie fixes the length of each output and the duration of each tool call to their recorded values. Later requests carry these outputs and the recorded tool results, thereby preserving the prefix that each request shares with earlier requests.

Models and deployment. We serve GLM-5.2 [18] for Putnam and CC and Kimi K2.6 [26] for OSWorld and for the run with a shifting workload composition. Each model is quantized to NVFP4 with an FP8 KV cache and uses multi-token prediction (MTP) for speculative decoding. It runs with four-way tensor parallelism (TP4) on nine 4×GB300 nodes [39]. On each node, host memory (L2) caches the KV of 4× as many tokens as GPU memory (L1).

Evaluation settings. Evolving Infra uses a planning window of $T_{\text{win}} = 600$ s with $m = 10$ slices and learns its iteration-time models online (Section 5.1). Appendix E.3 lists the other parameters of our runs. Each run starts with a 30-minute warmup, which we exclude from measurement. After the warmup, stationary runs serve one agent workload for one hour at a fixed concurrency, replacing each finished agent with a new one. We test three concurrency levels per workload, namely 224, 256, and 288 agents for Putnam, 90, 108, and 126 for CC, and 128, 144, and 160 for OSWorld. The run with a shifting workload composition serves 288 concurrent Putnam agents, 144 Putnam and 72 OSWorld agents, and 144 OSWorld agents in three consecutive hours after the warmup, shifting demand toward prefill.

Baselines. We compare Evolving Infra with Dynamo v1.5.1 [40] in the configuration of Section 3. To our knowledge, Dynamo is the state-of-the-art open-source system for dynamic PD disaggregation. Both start from the prefill-heavy split 6P3D, a common assumption for agentic workloads [4]. For the shifting workload composition, the ablation (Section 6.4), and the lowest concurrency of each stationary workload (Appendix D.1), we also compare with the best static split, which runs SGLang PD disaggregation at the fixed split with the highest throughput over the whole run [48]. All baselines use the SGLang Model Gateway [58] with cache-aware prefill routing and round-robin decode routing, the setup with the lowest P95 TTFT in Section 3.3. Each node runs HiCache [72] with LRU eviction and the same deduplicated host memory as Evolving Infra (Appendix A.5). We do not compare with the runtime role switch of SGLang [55], because it rejects speculative decoding, which both models use. Llumnix [32] cannot run on the Arm CPUs of our GB300 nodes, and Appendix A.1 compares it with Evolving Infra on H200 GPUs.



(a) Initial PD split (b) Online adaptation (c) Model convergence

Figure 8: **Evolving Infra achieves similar latency from every initial PD split, online phase-split adaptation costs almost no throughput, and the iteration-time models converge within a minute.** CC with 126 concurrent agents. (a) Mean E2E latency from each initial split. (b) Throughput at the best static split, 4P5D. (c) Mean error of the iteration-time models against learning time.

6.2 Stationary workloads

Low-cost online adaptation across agent workloads. Evolving Infra outperforms Dynamo in throughput, lost KV reuse, and mean E2E latency for every agent workload (Figure 6). It achieves **1.10**–**2.73**× the throughput of Dynamo (Appendix C.4), reduces lost KV reuse by **81**–**95%**, and lowers the mean E2E latency by **1.15**–**4.12**× and the P95 TTFT by up to **13.92**×. At the lowest concurrency of each workload, it also achieves 0.98–1.38× the throughput of the best static split under SGLang (Appendix D.1). Averaged over all nine settings, a phase switch of Evolving Infra completes in **6.2** s, including the wait for the running requests of the node, which the node keeps serving. This is over **100**× shorter than a Dynamo relaunch, which takes 754 s on average and keeps the node out of service throughout (Appendix C.1). The symmetric engine (Section 4.1) and the shared KV pool (Section 4.2) thus avoid the relaunch and retain the KV of the switching node, the two costs of a phase switch in Section 3.1. They stay fast when prefill and decode use different parallelism (Appendix D.4).

Accurate compute allocation for diverse agent workloads. Evolving Infra reduces the mean E2E latency by **3.80**–**4.12**× on CC, **2.34**–**2.69**× on OSWorld, and **1.15**–**1.23**× on Putnam relative to Dynamo (middle row of Figure 6). The planner (Section 5.2) contributes to this gain by matching the PD split to the compute demand of each workload purely online. It selects 3P6D–4P5D for CC and 8P1D for OSWorld (Appendix C.2). These splits differ from the best static splits under SGLang in Figure 2a, 4P5D and 6P3D, since the symmetric engines shift the best split (Appendix D.1). On the decode-heavy Putnam, the planner keeps only one or two prefill nodes to maximize throughput. This choice trades P95 TTFT for throughput, raising the P95 TTFT above that of Dynamo at 224 and 288 agents while lowering the mean E2E latency. Unlike the coarse signals in Section 3.2, the iteration-time models separate cached from uncached tokens (Section 5.1), and the average work of a request barely changes during short bursts. Evolving Infra thus switches only **2.0** times per run on average, whereas the Dynamo planner reacts to queue depth and KV utilization, switches **3.6** times per run, and deviates from the best static split of each workload for **41**–**100%** of the measurement period (Appendix C.2). Under open-loop arrivals, the planner also gives prefill more nodes as agents arrive faster, and Evolving Infra sustains twice the arrival rate of the best static split (Appendix D.3).

Accurate KV cache management under agent context management. Since asynchronous fetches make moving KV fast (Section 4.2), the router routes each request to the node with the shortest predicted TTFT (Section 5.3) rather than to the few nodes holding its prefix (Section 3.3). Evolving Infra thus reduces the P95 TTFT by **3.25**–**13.92**× on CC and OSWorld relative to Dynamo (bottom row of Figure 6). On CC, the most loaded prefill node queues requests **1.75**–**1.85**× as long as the average node, versus **2.29**–**2.61**× under cache-aware routing (Appendix C.3). With the counts of the directory (Section 4.3), the eviction of Evolving Infra (Section 5.3) retains nearly all the KV that agents still need (top row of Figure 6). In contrast, the LRU eviction of Dynamo drops the KV of branches that wait for tools, sub-agents, or users, which accounts for **34**–**46%** of its lost KV reuse on CC. The fraction of prompt tokens that Dynamo prefills without cached KV is up to **2.99**× that of Evolving Infra (Appendix C.1).

6.3 Shifting workload composition

Precise and timely adaptation to a shifting workload composition. The PD split of Evolving Infra tracks the best split of each 10-minute window as the planner (Section 5.2) follows the shift from Putnam to OSWorld (Figure 7a). In the OSWorld period, it settles at 8P1D, one prefill node more than the best split under SGLang, since the symmetric engines shift the best split (Appendix D.1). The best static split balances prefill and decode only in the mixed period (Figure 2b). The Dynamo planner keeps fewer prefill nodes than the best split for most of the run, as Section 3.2 predicts. Evolving Infra thus reduces the mean E2E latency by **2.15**× and **1.33**× and the P95 TTFT by **3.58**× and **1.88**× relative to Dynamo and the best static split, respectively (Figure 7b, c). It also achieves **2.14**× the throughput of Dynamo and **1.28**× that of the best static split, and the highest throughput in every period (Appendix C.4).

6.4 Ablation study

Sensitivity to the initial PD split. Evolving Infra achieves a similar mean E2E latency from every initial split (Figure 8a). Serving CC with 126 concurrent agents from four initial splits, from 3P6D to 6P3D, its mean E2E latency varies by at most **1.8%**. In all four runs, the planner converges to 4P5D (Appendix C.2). On this workload, online phase-split adaptation thus does not need a well-chosen initial split.

Contribution of each component. On CC with 126 concurrent agents, the infrastructure of Section 4 and the KV-reuse-aware eviction and routing of Section 5.3 alone achieve **1.37**× the throughput of the best static split at the same 4P5D split, without online phase-split adaptation (Figure 8b). With the routing of SGLang, asynchronous KV fetching alone achieves **1.12**× the throughput of upstream SGLang, because it cuts the idle time of prefill nodes from 23.6% to 14.4% (Appendix D.6). KV-reuse-aware routing and eviction contribute the rest. Compared with cache-aware routing at the same split, Evolving Infra loses 0.7% instead of 2.4% of the KV reuse and prefills 31% fewer uncached prompt tokens per request (Appendix D.5). Pausing agents as ThunderAgent does brings no such gain (0.95×), because KV capacity does not limit serving on GB300 nodes (Appendix D.5).

Cost of online phase-split adaptation. Enabling online phase-split adaptation at the best static split lowers the throughput by only **0.3%** (Figure 8b). Online adaptation thus costs almost no throughput even when the initial split is already the best. It also avoids offline profiling, which takes 10,752 GPU-hours for the three-hour CC trace on 256 GPUs even with a binary search and recurs for every agent workload (Appendix D.2). Online fitting is fast as well. The iteration-time models converge to a low mean error within a minute of learning (Figure 8c), far shorter than the 30-minute warmup of our runs.

7 Conclusions

We present Evolving Infra, an agent-adaptive serving framework that decouples compute allocation from KV cache management. Existing PD-disaggregated systems tie the KV cache of each node to the phase of that node. Changing the PD split thus either relaunches a node or loses its KV. Keeping KV on its node also forces routing to trade load balance against cache locality. In Evolving Infra, symmetric engines switch the phase of a node in place. A shared KV pool keeps the KV of every node readable by all other nodes.

On this data plane, Evolving Infra adapts compute allocation to diverse agent workloads and their shifting composition, and KV cache management to agent context management. Its planner reassigns nodes between prefill and decode with iteration-time models that separate cached from uncached tokens and are fitted online. Its router balances predicted TTFT across prefill nodes. With the branch counts of the KV directory, each node evicts first the KV that no active branch references, such as replaced histories and the contexts of finished sub-agents.

On 36 GB300 GPUs, a phase switch of Evolving Infra takes 6.2 s on average, over 100× shorter than a Dynamo relaunch. Under stationary load, Evolving Infra delivers up to 2.73× the throughput of Dynamo, loses 81–95% less KV reuse, and reduces the mean E2E latency by up to 4.12×. Under a shifting workload composition, it reduces the mean E2E latency by 2.15× and 1.33× relative to Dynamo and the best static split, respectively. These gains hold on 256 GB300 GPUs and, in simulation, on 3,072 GPUs (Appendix D.2).

References

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 117–134. USENIX Association, 2024.

[2] Itay Alroy. Achieving near-zero-downtime scaling in elastic expert parallelism. https://vllm.ai/blog/2026-10-19-elastic-ep-near-zero-downtime, 2026. vLLM Blog, Oct. 19, 2026. Accessed 2026-10.

[3] Itay Alroy, Yongji Wu, Rui Qiao, Tyler Michael Smith, Moein Khazraee, Omri Kahalon, Tzu-Ling Kan, and Ron Tourgeman. Elastic expert parallelism in vLLM. https://vllm.ai/blog/2026-05-14-elastic-expert-parallelism, 2026. vLLM Blog, May 14, 2026. Accessed 2026-10.

[4] Alex Angus, Will Van Eaton, and Dan Fu. Benchmarking inference at scale: Coding agents. https://www.together.ai/blog/coding-agent-benchmarks, May 2026. Together AI blog. Accessed: 2026-10-26.

[5] Anthropic. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku. https://www.anthropic.com/news/3-5-models-and-computer-use, 2024. Oct. 22, 2024. Accessed 2026-10.

[6] Anthropic. Compaction overview. https://platform.claude.com/docs/en/build-with-claude/compaction, 2026. Claude API documentation. Accessed 2026-10.

[7] Anthropic. Create custom subagents. https://code.claude.com/docs/en/sub-agents, 2026. Claude Code documentation. Accessed 2026-10.

[8] Applied Compute. trie: Trace replay inference evaluation. https://github.com/Applied-Compute/trie, 2026. GitHub repository. Accessed 2026-10.

[9] Malika Aubakirova, Alex Atallah, Chris Clark, Justin Summerville, and Anjney Midha. State of AI: An empirical 100 trillion token study with OpenRouter. *arXiv preprint arXiv:2601.10088*, 2026.

[10] Kyle Aubrey and Nick Stam. Inside NVIDIA Blackwell Ultra: The chip powering the AI factory era. https://developer.nvidia.com/blog/inside-nvidia-blackwell-ultra-the-chip-powering-the-ai-factory-era/, 2025. NVIDIA Technical Blog, August 22, 2025. Accessed 2026-10.

[11] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[12] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[13] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[14] DeepSeek-AI. DeepSeek-V3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.

[15] DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026.

[16] Yicheng Feng, Xin Tan, Yangtao Deng, Yimin Jiang, Yibo Zhu, and Hong Xu. Frontier: Towards comprehensive and accurate LLM inference simulation. *arXiv preprint arXiv:2605.21312*, 2026.

[17] Kaihua Fu, Yukun Zhou, Chaokun Chang, Yinghao Yu, Luping Wang, Guodong Yang, Jiuchen Shi, Quan Chen, and Wei Wang. Capture the lifecycle: KV cache management in ReAct agents with KVTether. *arXiv preprint arXiv:2609.39819*, 2026.

[18] GLM-5-Team. GLM-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.

[19] Ke Hong, Lufang Chen, Zhong Wang, Xiuhong Li, Qiuli Mao, Jianping Ma, Chao Xiong, Guanyu Wu, Buhe Han, Guohao Dai, Yun Liang, and Yu Wang. semi-PD: Towards efficient LLM serving via phase-wise disaggregated computation and unified storage. *arXiv preprint arXiv:2504.19867*, 2025.

[20] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. MemServe: Context caching for disaggregated LLM serving with elastic memory pool. *arXiv preprint arXiv:2406.17565*, 2024.

[21] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.

[22] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[23] Hao Kang, Ziyang Li, Weili Xu, Xinyu Yang, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. ThunderAgent: A fast, simple, and program-aware agentic inference system. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pp. 55630–55657, 2026.

[24] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[25] Kimi Team. Kimi K2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.

[26] Kimi Team. Kimi K2.6. https://huggingface.co/moonshotai/Kimi-K2.6, April 2026. Model card, Moonshot AI. Accessed: 2026-10-26.

[27] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. *arXiv preprint arXiv:2511.02230*, 2025.

[28] Zongze Li, Jingyu Liu, Zach Xu, Yineng Zhang, Tahseen Rabbani, and Ce Zhang. Not all prefills are equal: PPD disaggregation for multi-turn LLM serving. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pp. 68755–68771, 2026.

[29] John D. C. Little. A proof for the queuing formula: $L = \lambda W$. *Operations Research*, 9(3):383–387, 1961. doi: 10.1287/opre.9.3.383.

[30] Banruo Liu, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, and Esha Choukse. Agentic coding in the wild: Characterizing GitHub Copilot traces at production scale. *arXiv preprint arXiv:2608.00101*, 2026.

[31] Tongxuan Liu, Tao Peng, Peijun Yang, Xiaoyang Zhao, Xiusheng Lu, Weizhe Huang, Zirui Liu, Xiaoyu Chen, Zhiwei Liang, Jun Xiong, Donghe Jin, Minchao Zhang, Jinrong Guo, Yingxu Deng, Xu Zhang, Xianzhe Dong, Siqi Wang, Siyu Wu, Yu Wu, Zihan Tang, Yuting Zeng, Yanshu Wang, Jinguang Liu, Meng Kang, Menxin Li, Yunlong Wang, Yiming Liu, Xiaolong Ma, Yifan Wang, Yichen Zhang, Jinrun Yin, Keyang Zheng, Jiawei Yin, Jun Zhang, Ziyue Wang, Xiaobo Lin, Liangyu Liu, Liwei Lan, Yang Liu, Chunhua Peng, Han Liu, Songcheng Ren, Xuezhu Wang, Yunheng Shen, Yi Wang, Guyue Liu, Yitao Hu, Hui Chen, Tong Yang, Hailong Yang, Jing Li, Guiguang Ding, and Ke Zhang. xLLM technical report. *arXiv preprint arXiv:2510.14686*, 2025.

[32] Llumnix Project. Adaptive PD scheduling. https://docs.llumnix.ai/design/scheduler/adaptive_pd_scheduling.html, 2026. Llumnix documentation. Accessed 2026-10.

[33] Llumnix Project. SLO-aware scheduling. https://docs.llumnix.ai/design/scheduler/slo_aware_scheduling.html, 2026. Llumnix documentation. Accessed 2026-10.

[34] LMCache Project. [VLLM|CacheEngine]support only save on rank 0 for DS MLA. https://github.com/LMCache/LMCache/pull/1128, 2025. GitHub pull request #1128, merged Aug. 13, 2025. Accessed 2026-10.

[35] Mooncake Project. SGLang HiCache x Mooncake Store performance. https://github.com/kvcache-ai/Mooncake/blob/main/docs/source/performance/sglang/sglang-hicache-benchmark-results-v1.md, 2026. Accessed 2026-10.

[36] Mooncake Project. Mooncake store. https://github.com/kvcache-ai/Mooncake/blob/main/docs/source/design/store/mooncake-store.md, 2026. Accessed 2026-10.

[37] Mooncake Project. Mooncake transfer engine. https://github.com/kvcache-ai/Mooncake/blob/main/docs/source/design/transfer-engine/index.md, 2026. Accessed 2026-10.

[38] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

[39] NVIDIA. NVIDIA GB300 NVL72. https://www.nvidia.com/en-us/data-center/gb300-nvl72/, 2025. Product page. Accessed 2026-10.

[40] NVIDIA. NVIDIA Dynamo: A datacenter scale distributed inference serving framework. https://github.com/ai-dynamo/dynamo, 2026. Version v1.5.1. Accessed 2026-10.

[41] NVIDIA. Publish KV events from a custom backend. https://docs.nvidia.com/dynamo/v1.5.1/advanced-customizations/writing-custom-backends/publish-kv-events, 2026. NVIDIA Dynamo documentation, v1.5.1. Accessed 2026-10.

[42] NVIDIA. Offloading support matrix. https://docs.nvidia.com/dynamo/v1.5.1/knowledge-base/modular-components/router/offloading-support-matrix, 2026. NVIDIA Dynamo documentation, v1.5.1. Accessed 2026-10.

[43] NVIDIA. Planner. https://docs.nvidia.com/dynamo/v1.5.1/knowledge-base/modular-components/planner/overview, 2026. NVIDIA Dynamo documentation, v1.5.1. Accessed 2026-10.

[44] NVIDIA. Router design. https://docs.nvidia.com/dynamo/v1.5.1/knowledge-base/modular-components/router/router-design, 2026. NVIDIA Dynamo documentation, v1.5.1. Accessed 2026-10.

[45] NVIDIA. Routing concepts. https://docs.nvidia.com/dynamo/v1.5.1/knowledge-base/modular-components/router/routing-concepts, 2026. NVIDIA Dynamo documentation, v1.5.1. Accessed 2026-10.

[46] NVIDIA. SGLang for agentic workloads. https://docs.nvidia.com/dynamo/v1.5.1/knowledge-base/modular-components/backends/sg-lang/agents-on-sg-lang, 2026. NVIDIA Dynamo documentation, v1.5.1. Accessed 2026-10.

[47] OpenAI. Building more with GPT-5.1-Codex-Max. https://openai.com/index/gpt-5-1-codex-max/, 2025. Nov. 19, 2025. Accessed 2026-10.

[48] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 118–132, 2024. doi: 10.1109/ISCA59077.2024.00019.

[49] Yifan Qiao, Trong Dao Le, Ao Shen, Zhewen Li, and Bowen Wang. Serving agentic workloads at scale with vLLM x Mooncake. https://vllm.ai/blog/2026-05-06-mooncake-store, 2026. vLLM Blog, May 6, 2026. Accessed 2026-10.

[50] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pp. 155–170. USENIX Association, 2025.

[51] SemiAnalysis. CC Traces — Weka, With Subagents, v7 only (Jun 21 2026). https://huggingface.co/datasets/semianalysisai/cc-traces-weka-062126, 2026. Hugging Face dataset. Accessed 2026-10.

[52] SGLang Project. HiCache system design and optimization. https://docs.sglang.io/docs/advanced_features/hicache_design, 2026. Accessed 2026-10.

[53] SGLang Project. PD disaggregation. https://docs.sglang.io/docs/advanced_features/pd_disaggregation, 2026. Accessed 2026-10.

[54] SGLang Project. Radix cache eviction policies. https://docs.sglang.io/docs/advanced_features/radix_eviction_policy, 2026. Accessed 2026-10.

[55] SGLang Project. Introduce runtime role switching between prefill and decode. https://github.com/sgl-project/sglang/pull/28403, 2026. GitHub pull request #28403, merged Sep. 17, 2026. Accessed 2026-10.

[56] SGLang Project. Server arguments. https://docs.sglang.io/docs/advanced_features/server_arguments, 2026. SGLang documentation. Accessed 2026-10.

[57] SGLang Project. SGLang source code. https://github.com/sgl-project/sglang, 2026. Tree at commit 8080507: `python/sglang/srt/mem_cache/unified_radix_cache.py` (prefetch stop policies) and `python/sglang/srt/arg_groups/fields/memory.py` (`--enable-unified-cache-external-linker`), `python/sglang/srt/mem_cache/utils.py` (page hashes), `python/sglang/srt/managers/cache_controller.py` (background backup), and `python/sglang/srt/managers/scheduler.py` (prefetch on enqueue). Accessed 2026-10-30.

[58] SGLang Team. SGLang Model Gateway. https://docs.sglang.io/docs/advanced_features/sgl_model_gateway, 2026. SGLang documentation (release v0.5.21). Accessed 2026-10.

[59] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[60] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 173–191. USENIX Association, 2024.

[61] The SGLang Team. Deploying DeepSeek with PD disaggregation and large-scale expert parallelism on 96 H100 GPUs. https://lmsys.org/blog/2025-05-05-large-scale-ep/, 2025. LMSYS Org blog, May 5, 2025. Accessed 2026-10.

[62] George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. PutnamBench: Evaluating neural theorem-provers on the Putnam mathematical competition. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*, pp. 11545–11569, 2024.

[63] vLLM Project. Deduplicate replicated MLA KV in the shared CPU region. https://github.com/vllm-project/vllm/pull/48906, 2026. GitHub pull request #48906, merged Jul. 26, 2026. Accessed 2026-10.

[64] vLLM Project. NixlConnector usage guide. https://docs.vllm.ai/en/latest/features/nixl_connector_usage.html, 2026. vLLM documentation. Accessed 2026-10.

[65] vLLM Team. Data parallel deployment. https://docs.vllm.ai/en/latest/serving/data_parallel_deployment.html, 2026. vLLM documentation. Accessed 2026-10.

[66] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. KVCache cache in the wild: Characterizing and optimizing KVCache cache at a large cloud provider. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pp. 465–482, 2025.

[67] Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. BrowseComp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.

[68] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

[69] Yu Wu, Tongxuan Liu, Yuting Zeng, Siyu Wu, Jun Xiong, Xianzhe Dong, Hailong Yang, Ke Zhang, and Jing Li. Arrow: Adaptive scheduling mechanisms for disaggregated LLM inference architecture. *arXiv preprint arXiv:2505.11916*, 2025.

[70] Yuxing Xiang, Xue Li, Kun Qian, Yan Zhang, Wenyuan Yu, Ennan Zhai, Xin Jin, and Jingren Zhou. ServeGen: Workload characterization and generation of large language model serving in production. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, 2026.

[71] Ao Xiao, Bangzheng He, Baoquan Zhang, Baoxing Huai, Bingji Wang, Bo Wang, Bo Xu, Boyi Hou, Chan Yang, Changhong Liu, Cheng Cui, Chenyu Zhu, Cong Feng, Daohui Wang, Dayun Lin, Duo Zhao, Fengshao Zou, Fu Wang, Gangqiang Zhang, Gengyuan Dan, Guanjie Chen, Guodong Guan, Guodong Yang, Haifeng Li, Haipei Zhu, Haley Li, Hao Feng, Hao Huang, Hao Xu, Hengrui Ma, Hengtao Fan, Hui Liu, Jia Li, Jiang Liu, Jiang Xu, Jie Meng, Jinhan Xin, Junhao Hu, Juwei Chen, Lan Yu, Lanxin Miao, Liang Liu, Linan Jing, Lu Zhou, Meina Han, Mingkun Deng, Mingyu Deng, Naitian Deng, Nizhong Lin, Peihan Zhao, Peng Pan, Pengfei Shen, Ping Li, Qi Zhang, Qian Wang, Qin Zhang, Qingrong Xia, Qingyi Zhang, Qunchao Fu, Ren Guo, Ruimin Gao, Shaochun Li, Sheng Long, Shentian Li, Shining Wan, Shuai Shen, Shuangfu Zeng, Shuming Jing, Siqi Yang, Song Zhang, Tao Xu, Tianlin Du, Ting Chen, Wanxu Wu, Wei Jiang, Weinan Tong, Weiwei Chen, Wen Peng, Wenli Zhou, Wenquan Yang, Wenxin Liang, Xiang Liu, Xiaoli Zhou, Xin Jin, Xinyu Duan, Xu Li, Xu Zhang, Xusheng Chen, Yalong Shan, Yang Gan, Yao Lu, Yi Deng, Yi Zheng, Ying Xiong, Yingfei Zheng, Yiyun Zheng, Yizhou Shan, Yong Gao, Yong Zhang, Yongqiang Yang, Yuanjin Gong, Yue Yu, Yuetao Chen, Yukun Zhu, Yulong He, Yusu Zhao, Yuyan Wu, Zenan Zhang, Zhaojin Zhuo, Zhaoyang Ji, Zhefeng Wang, Zheng Wang, Zhenan Fan, Zhenhua Yang, Zhenli Sheng, Zhibin Yu, Zhigang Ji, Zhihao Ren, Zhipeng Bian, Zhixia Liu, Zhiyu Dong, Zhonghua Li, Zhou Yu, Zhuoming Shen, Zhuwei Peng, Zi Ye, Zihao Xiang, Zimin Fu, and Zixuan Zhang. Huawei cloud model-as-a-service on the CloudMatrix384 SuperPod. *arXiv preprint arXiv:2508.02520*, 2025.

[72] Zhiqiang Xie. SGLang HiCache: Fast hierarchical KV caching with your favorite storage backends. https://lmsys.org/blog/2025-09-10-sglang-hicache, 2025. LMSYS Blog, Sep. 10, 2025. Accessed 2026-10.

[73] xLLM Team. xLLM-Service. https://github.com/xLLM-AI/xllm-service, 2026. Tree at commit a1251f4, `xllm_service/scheduler/managers/instance_mgr.cpp`. Accessed: 2026-10-30.

[74] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, pp. 50528–50652, 2024.

[75] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025.

[76] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pp. 521–538. USENIX Association, 2022.

[77] Mengqi Yuan, Zilong Zhou, Xinzhuang Xiong, Weiming Wu, Jiayang Sun, Jiamin Song, Kaiqian Cui, Bowen Wang, Haoyuan Wu, Yitong Li, Dunjie Lu, Haikong Lu, Qi Zhen, Xinyuan Wang, Jiaqi Deng, Yuhao Yang, Cheng Chen, Boyuan Zheng, Alex Su, Xiao Yu, Hao Zou, Saaket Agashe, Xing Han Lu, Manpreet Kaur, Zhengyang Qi, Vincent Sunn Chen, Frederic Sala, Dayiheng Liu, Junyang Lin, Zhou Yu, Yu Su, Siva Reddy, Xin Eric Wang, Peng Qi, Tianbao Xie, and Tao Yu. OSWorld 2.0: Benchmarking computer use agents on long-horizon real-world tasks. *arXiv preprint arXiv:2606.29537*, 2026.

[78] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pp. 62557–62583, 2024. doi: 10.52202/079017-2000.

[79] Yihao Zheng, Jingzhe Jiang, Dejiang Zhu, Zhiyuan Tan, Yang Tian, Tao Wang, and Minchen Yu. Semantics, workflows, and infrastructure: Understanding agent serving at production scale. *arXiv preprint arXiv:2609.34432*, 2026.

[80] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 193–210. USENIX Association, 2024.

A Additional Related Work

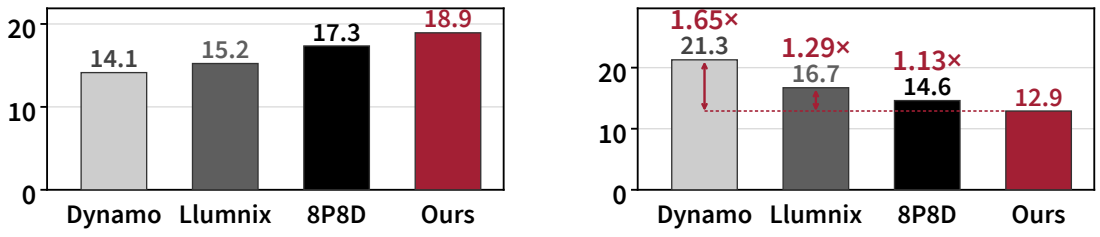
This appendix extends the review of Section 2.2. Appendix A.1 details how existing systems change the PD split. Appendix A.2 and Appendix A.3 describe the KV tiers and the KV index that Sections 4.2 and 4.3 build on. Appendix A.4 compares agent-aware KV management with Evolving Infra, and Appendix A.5 explains why host memory can hold the KV that agents reuse.

A.1 Prefill–decode split adaptation

Current serving engines fix their phase at launch by default. Systems that adapt the PD split around this limit either relaunch engines, which takes minutes and empties their KV caches, or switch them in place. Those that switch in place lose the KV of a switched engine or size the pools with a coarse cost model. Others keep the roles of nodes fixed and shift only compute or KV between the phases, inside an engine or between fixed prefill and decode nodes.

vLLM and SGLang relaunch to change phase. In SGLang, a flag at launch makes each engine a prefill or a decode engine, and the router is given separate lists of prefill and decode engines [53]. The two roles run different scheduling loops with role-specific state. A prefill engine keeps a bootstrap queue and sends KV to decode engines. A decode engine reserves KV slots for each incoming request before its prefill finishes, receives KV through its own threads and queues, and captures CUDA graphs for decoding [53, 55]. vLLM likewise configures each engine at launch to produce KV for prefill or to consume it for decode, and its proxy is given fixed lists of prefill and decode engines [64]. Changing the phase of an engine thus requires relaunching it with a new configuration. SGLang's recent opt-in runtime switch avoids the relaunch, but it still drains the engine to idle and releases its prefix cache [55]. Systems built on these engines therefore relaunch them. The Dynamo planner changes the phase of a node by terminating its worker, without waiting for in-flight requests, and launching a worker of the other phase [43]. The new worker must load the model and starts with an empty KV cache, and Dynamo notes that spinning up a GPU worker takes minutes [43]. In the runs of Section 6.2, a Dynamo switch takes 754 s on average, from the planner decision until the relaunched worker is ready, and on CC with 108 agents Dynamo loses 4.9% of the available KV reuse over its four switches (Appendix C.1). While a node relaunches, only eight of the nine nodes serve requests, which covers 44–91% of the one-hour measurement period (Appendix C.2).

Arrow switches in place but loses KV. Arrow [69] keeps nodes stateless, so that any node can serve both prefill and decode requests, and it moves nodes between the prefill and decode pools without a restart. Its code is not public, so Section 3 evaluates our reimplementation of its planner and scheduler on the symmetric engines of Evolving Infra. Arrow has three problems on agentic workloads. It sizes the pools by TTFT and TPOT, which agent context management makes fluctuate. It predicts TTFT from the full input length,



(a) Throughput (req/s)

(b) Mean E2E latency (s)

Figure A.1: **Dynamo and Llumnix adapt the PD split but fall behind the best static split.** DeepSeek-V4-Flash runs on 16 engines of four H200 GPUs each (TP4) and serves CC at 384 concurrent sessions. Dynamo, Llumnix and Evolving Infra start from 12P4D. Of the two static splits we ran, 8P8D and 9P7D, 8P8D is the best static split. (a) Both baselines deliver less throughput than the best static split, which Evolving Infra exceeds by 9%. (b) The red number above a baseline bar gives its ratio to Evolving Infra.

which ignores KV reuse. Its scheduler also ignores where KV is cached, and each node keeps its KV in its own cache, so later prefills do not reuse the KV of a node that moves to the decode pool. In our runs on CC, Arrow switches nodes 34 times, ends at 8P1D, and delivers 65% less throughput than the best static split under SGLang PD disaggregation (Section 3.2). In these runs, it loses 55.5% of the available KV reuse (Section 3.1).

xLLM cannot switch phases and route by cached KV at once. xLLM [31] switches nodes between phases in place, but its open-source service implements this switching and KV-aware routing as alternative policies chosen at launch [73]. Only its SLO-aware policy switches phases, and it flips single nodes in reaction to individual requests instead of sizing the split with a planner. This policy predicts TTFT from the full prompt length, so it counts cached prefixes as prefill work and ignores where KV is cached. Its cache-aware policy routes by prefix hits but never switches phases.

Llumnix adapts the split with a coarse cost model. Llumnix [60] reschedules running requests across engines by live-migrating each request together with its KV, which balances load, reduces memory fragmentation, and isolates high-priority requests. Its adaptive PD mode assigns phases to engines at runtime to meet TTFT and TPOT targets [32], using a cost model that predicts TTFT by interpolating between profiled points by the number of prefill tokens and the decode load [33]. A single token count cannot separate the uncached tokens of an agent request from the long cached prefix they attend to. Part of its KV transfer stack is also released only as a pre-built binary without source code, so we could not run Llumnix on the Arm CPUs of our GB300 nodes. In our runs on H200 GPUs, Llumnix delivers 12.2% less throughput than the best static split (Figure A.1).

semi-PD shares KV between the phases only inside an engine. semi-PD [19] runs a prefill process and a decode process on every GPU of an engine, divides the SMs of each GPU between them through NVIDIA MPS, and keeps one copy of the weights and the KV, its unified storage, in the GPU memory of the engine. Shifting SMs restarts only these processes, while a resident process keeps the weights and the KV, so semi-PD avoids costly phase switches inside an engine. Its controller, however, shifts SMs to the phase whose TTFT or TPOT misses its target, and the prefill bursts of agents, such as sub-agents that start together, perturb both (Section 3.2). Across engines, its router weighs the KV cache hit rate of each engine against GPU utilization and queue depth, the trade-off between cache locality and load balance of Section 3.3.

MemServe and PPD reuse multi-turn KV but keep the split fixed. MemServe [20] and PPD [28] let the later turns of a conversation reuse the KV that its previous turn left on the decode side. MemServe moves this KV to the prefill engine: the decode engine sends the KV that it produced back after each request, and a global scheduler routes each request by where its prefix is cached. PPD instead moves the prefill to the KV: it prefills the uncached tokens of a later turn on the decode node that holds the KV of the conversation, which slows decoding far less than a full prefill does, and a table profiled offline decides per request whether to take this path. Neither, however, adapts the split: MemServe's engines are prefill-only, decode-only, or colocated, and PPD restarts its engines to change the split, so neither follows the shifting workload composition (Figure 2b). PPD's offline table, as its authors note, becomes suboptimal when the workload drifts far from the one it was profiled on. MemServe also judges KV reuse from stale records: its scheduler learns where KV resides only from the responses that pass through it, so it misses evictions and expires its records after a fixed time-to-live. PPD keeps the KV of a conversation on one decode node, so each later turn either runs on that node or recomputes the previous response on a prefill node.

Evolving Infra instead switches the phase of a node in place, and the shared KV pool keeps the KV of the node readable by all other nodes (Sections 4.1 and 4.2).

A.2 KV memory hierarchy in current serving engines

Section 4.2 builds the shared KV pool of Evolving Infra on SGLang HiCache. This subsection describes how current engines tier KV, how they share it across nodes through Mooncake, and why this sharing adds latency.

Private host tiers. SGLang HiCache backs the GPU KV cache (L1) of an engine with host memory (L2) and an optional storage backend (L3) such as Mooncake Store [52, 72]. L1 and L2 are private to each engine, and every tensor-parallel rank allocates its own L2 pool. Engines therefore never read each other's L2: KV that one engine computes becomes visible to another only after it reaches L3 [52].

Sharing through Mooncake. Mooncake Store pools the DRAM that nodes contribute into a cluster-wide object store. A master service keeps the metadata of each object, and clients move the data directly over RDMA with the Mooncake Transfer Engine [36, 37]. A put first asks the master to allocate space and then commits the object, and a get first asks the master where the object is stored. SGLang backs up pages from L2 to Mooncake in the background, keyed by a hash of the prefix that ends at each page. When a request is queued, an engine asks Mooncake which pages of its prefix exist, prefetches them into its own L2, and loads them into GPU memory layer by layer during prefill [52, 57]. vLLM's Mooncake store connector instead registers GPU memory with Mooncake and moves KV blocks directly between GPU memory and the pool [49], and a newer SGLang option links its radix cache to Mooncake in the same way, without HiCache's host tier [57].

Why fetching through the pool adds latency. KV takes extra hops. With HiCache, it moves from the L2 of the producing engine into a Mooncake segment, which may lie on a third node, and from there into the L2 of the consuming engine before it reaches the GPU. Every lookup, put, and get also involves RPCs to the master, and a page becomes visible only after its backup commits [36]. The prefetch starts when the request is queued but fills only L2, and the GPU loads the pages layer by layer during prefill. If the prefetch has not finished when the prefill could start, SGLang stops it under the best-effort policy and recomputes the pages that have not arrived, waits for it under the wait-complete policy, and under the default timeout policy waits until a deadline and then recomputes the rest [52, 57]. Mooncake's own benchmark accordingly finds HiCache slightly faster with L2 alone than with Mooncake at equal hit rates in its first rounds, because L2 alone avoids fetching from remote storage [35]. The same KV can also occupy the L2 of the producer, the pool, and the L2 of each consumer at once. Evolving Infra instead lets prefill nodes read KV directly from the L2 of any peer over RDMA, restores it into GPU memory before the prefill begins, and keeps L2 across phase switches (Sections 4.1 and 4.2).

A.3 Dynamo KV indexer and router

The KV directory of Evolving Infra (Section 4.3) follows the KV indexer of the Dynamo router. This subsection describes how the index is built and updated, how the router scores nodes with it, and what it does not record.

Cluster-wide KV index. The KV router of Dynamo keeps a cluster-wide index of cached KV in a prefix tree [44]. Each entry of the tree stands for one KV block, keyed by a hash of its tokens, and records the nodes that hold the block. To route a request, the router walks the tree along the blocks of the request and finds, for every node, how many leading blocks it already holds.

KV events. Engines keep the tree current through KV events. SGLang and vLLM publish an event over ZMQ whenever they store or evict blocks, and a publisher on each node relays the events to the routers [41]. The engine identifies each block by a prefix hash over all tokens up to and including the block. When it stores new blocks, its event carries their tokens and prefix hashes and the prefix hash of their parent block. Through a per-node table from prefix hashes to tree entries, the router finds the entry of the parent block, descends from it along the token hashes of the new blocks, creating entries as needed, and adds the node to each of these entries [40, 41, 44]. When the engine evicts blocks, its event names them by their prefix hashes, and the router removes the node from their entries. With HiCache, SGLang also names the memory tier in each event, and the router tracks host memory and disk in separate indexes beside the GPU tree [40, 42].

Scoring. For each candidate node, the router adds the prompt blocks of the request to the prefill blocks already assigned to the node, subtracts a credit for each leading block that the node already holds, and adds

the blocks that the node would be decoding. The router tracks these loads itself from the requests that it routes, not from KV events, and sends the request to the node with the lowest cost [44]. A block held in GPU memory earns a full credit, while blocks in host memory and on disk earn 0.75 and 0.25 by default [42]. The host and disk credits are constants: they depend neither on how long loading the blocks takes nor on the running requests that the loading delays on the node. Evolving Infra instead predicts the TTFT of the request on each prefill node, including the part of a KV fetch that the wait in the queue does not hide (Section 5.3 and Appendix E.3).

What the index does not record. The index records where each block is cached, on which nodes and in which tier, but not which requests or agent branches will reuse it. KV events report only stores and evictions, not the forks, compactions, and finished sub-agents that decide this reuse. Eviction therefore stays with each engine, and SGLang evicts in LRU order by default [54]. Dynamo can also pass a per-request priority from the harness to SGLang, which can then evict the KV of the lowest-priority requests first [46]. Neither recency nor priority changes when compaction replaces a history or a KV transfer leaves a copy behind: these blocks keep their last access time, and a cached prefix keeps the highest priority among the requests that reached it [54]. Evolving Infra keeps the same event-driven tree and adds one attribute to each block, the number of active branches that reference it on each node (Section 4.3). The counts follow agent context management and KV transfers, and each node evicts the host-memory blocks with zero counts first (Section 5.3).

A.4 Agent-aware KV management and routers

This subsection compares Evolving Infra with systems that use the structure of agents to manage KV.

Program-aware scheduling. ThunderAgent [23] and Continuum [27] schedule the requests of an agent together so that its KV survives tool calls. Continuum pins the KV of a multi-turn program in GPU memory for a time-to-live that it estimates from the durations of past calls to the same tool. ThunderAgent abstracts each agent workflow as a program. When the programs on a data-parallel replica need more KV than its GPU cache holds, ThunderAgent pauses programs that wait for tools, evicts their KV, and resumes them later, possibly on another replica with free memory. Both systems thus target KV-cache thrashing, in which the KV of a waiting program is evicted from GPU memory and its whole context must be prefilled again. Both run prefill and decode on the same nodes, and both treat the context of a program as growing only by appending.

Thrashing on current models and hardware. Thrashing occurs only when programs need more KV than a replica can hold. Current engines back GPU memory with a host-memory tier, and compact KV representations together with deduplication let this tier hold several times as much KV as GPU memory (Appendix A.5). A context evicted from GPU memory then stays in host memory and is reloaded rather than recomputed. ThunderAgent found such offloading too slow over PCIe [23], but on GB300 nodes the reload runs over the NVLink-C2C link between the Grace CPUs and the GPUs [10, 39]. In our runs, Dynamo, whose SGLang engines evict host memory in LRU order, loses at most 7.8% of the available KV reuse, and Evolving Infra at most 1.0%, without pausing any agent (Appendix C.1). Host memory instead keeps KV that no active branch needs: the source copies that KV transfers leave, the histories that compaction replaces, and the contexts of finished sub-agents. Evolving Infra therefore counts the branches that reference each copy (Section 4.3) instead of pausing agents. Appendix D.5 compares Evolving Infra with ThunderAgent and cache-aware routing on GB300 nodes.

Agent-aware KV management. KVTether [17] exports runtime context operations from agent harnesses and maps them to KV lifecycle states, accounting for shared prefixes across contexts. These states guide cache admission, reclamation, and eviction in a multi-tier cache with cross-node transfers. Its presented design focuses on KV lifecycle management and does not address online adaptation of the PD split; its evaluation uses a fixed PD configuration.

A.5 Extending host memory with deduplication

Section 4.2 keeps the KV that agents reuse in host memory (L2), and Appendix A.4 argues that this tier largely absorbs KV thrashing. This subsection explains why L2 is large enough.

Compact KV. Although smaller than L3, L2 is sufficient for recent open models [18, 25]. These models increasingly adopt compact KV representations such as MQA [59] and MLA [13]. In MLA, all attention heads share one latent vector of 576 values per token and layer. With an FP8 KV cache, a token takes about 35 KB of KV in Kimi K2.6 and about 50 KB in GLM-5.2, including the keys of the sparse-attention indexer of GLM-5.2. Full multi-head attention at the same precision would need 36× and about 50× as much.

Deduplication across tensor-parallel ranks. Because all heads share the latent vector, tensor parallelism does not split MLA KV: every rank holds the complete KV of each token [52]. SGLang HiCache allocates its host pool per rank and thus stores the same KV once per GPU in L2, although only one rank writes it to the storage backend [52]. vLLM's CPU offloading instead keeps one copy of each replicated MLA block in a host region that all ranks share, so that at a tensor-parallel degree of N the host tier holds about N times as many blocks [63]. LMCache saves MLA KV from one rank and broadcasts it to the other ranks on retrieval [34]. Our engines deduplicate L2 in the same way: L2 stores a single copy per node for all attention heads and broadcasts it to all GPUs of the node when fetched in L1. All systems in our evaluation, including the baselines, use this deduplication. When serving GLM-5.2 on NVIDIA GB300, L2 typically holds **4**× as much KV as L1. A GB300 node has about 1 TB of host memory [39], which could hold about 20 million tokens of GLM-5.2 KV as a single copy, but only about 5 million if each of its four GPUs kept its own copy.

Production traces of a large cloud provider show that the cache size needed for an ideal hit ratio is moderate [66]. With a deduplicated host tier, capacity thus rarely forces recomputation in our runs (Appendix A.4), and the remaining waste lies in KV that no active branch references (Section 4.3).

B Extended Theoretical Analysis

This appendix extends the iteration-time models of Section 5.1. Appendix B.1 fits both models on another GPU and model. Appendix B.2 explains why the prefill iteration-time model holds for attention architectures other than full attention. Appendix B.3 derives $\phi(B)$, the fraction of routed experts that a decode iteration reads, and Appendix B.4 explains why the decode iteration-time model omits the context length.

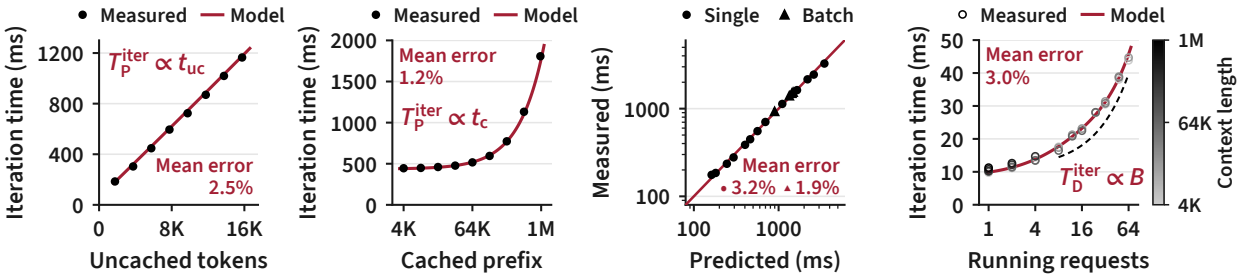
B.1 Iteration-time models on H200 with DeepSeek-V4-Flash

We repeat the measurement of Figure 5 on a second GPU and model to test whether the form of the iteration-time models holds beyond GB300 and GLM-5.2. The engine serves DeepSeek-V4-Flash [15] on four H200 GPUs with four-way tensor parallelism (TP4). DeepSeek-V4-Flash differs from GLM-5.2 in the two components that shape the models, attention and the MoE layers. Its attention combines a sliding window with compressed attention (Appendix B.2), and its MoE layers route each token to 6 of 256 experts instead of 8 of 256. Its speculative decoding feeds $t_{\text{fwd}} = 6$ tokens of each running request into every decode iteration. We measure the same settings as in Figure 5. As there, we fit the prefill model on 20 single-request settings and the decode model on the ten points with a 16K context.

Figure B.1 shows that both models track the measured iteration times as closely as on GB300. Outside the 20 fitted settings, the prefill model has a mean error of 2.6% on the 28 single-request settings and 1.9% on the 12 batches of 2–8 requests. No batch deviates by more than 4.1%, including the batches of eight requests at a 256K cached prefix. Outside the ten fitted points, the decode model has a mean error of 3.6%.

B.2 Prefill iteration-time model beyond full attention

Under full attention, each uncached token attends to all cached tokens and to the uncached tokens before it. A request with t_c cached tokens and t_{uc} uncached tokens thus computes $A = t_{uc}t_c + t_{uc}(t_{uc}+1)/2$ query–key pairs (Section 5.1). Many current models let an uncached token attend to only part of the earlier tokens. This



(a) Prefill iteration time vs. uncached tokens

(b) Prefill iteration time vs. cached prefix

(c) Predicted vs. measured iteration time

(d) Decode iteration time vs. running requests

Figure B.1: **The iteration-time models also closely track the iteration times of DeepSeek-V4-Flash on H200.** Points are GPU times per iteration of DeepSeek-V4-Flash on four H200 GPUs, and red lines are the models in Equations (1) and (2), fit on the same settings as in Figure 5. Each panel reports the mean error on the remaining points. (a) The prefill iteration time grows nearly linearly with the uncached prefill tokens at a 128K cached prefix. (b) It grows linearly with the cached prefix at 8K uncached prefill tokens (log-scale axis). (c) Outside the fitted settings, the mean error is 2.6% for single requests and 1.9% for batches of 2–8 requests. The panel shows 17 of the 60 measured prefill iterations, evenly spaced and including the four batches with the largest errors. (d) The decode iteration time follows the model as the running requests grow, and with 8 or more running requests, the context length (darker for longer) changes it by at most 6.9%.

subsection explains how these attention layers work and why the prefill iteration-time model in Equation (1) still expresses their time.

Sliding-window attention. A sliding window lets each token attend only to a fixed number of the most recent tokens [11, 21]. Once the context is longer than the window, every uncached token computes the same number of query–key pairs, independent of t_c. The attention time of a request thus grows in proportion to t_{uc}, and the term $c_{n,1} \sum t_{uc}$ in Equation (1) captures it.

Linear attention. Linear attention replaces the query–key pairs with earlier tokens by a state of fixed size [24, 75]. Each uncached token updates this state and computes its output from it, and the kernels process the uncached tokens in chunks of a fixed length. Every uncached token thus costs the same, independent of t_c. The attention time of a request grows in proportion to t_{uc}, and the term $c_{n,1} \sum t_{uc}$ in Equation (1) captures it.

DeepSeek Sparse Attention (DSA). DSA, which GLM-5.2 uses [14, 18], splits attention into two steps. A lightweight indexer first scores every earlier token of each uncached token, and the core attention then attends only to the 2048 tokens with the highest scores. Each score is one query–key pair, and the indexer thus computes A pairs per request, as full attention does, at a lower cost per pair. Its time grows with A, and the term $c_{n,2} \sum A$ in Equation (1) captures it. The core attention computes at most 2048 pairs per uncached token, and its time grows with t_{uc}, which the term $c_{n,1} \sum t_{uc}$ captures.

Compressed attention in DeepSeek-V4. Most layers of DeepSeek-V4 [15] combine a sliding window of 128 tokens with one of two compressed attentions. Both compress the keys and values of consecutive tokens into entries, and an uncached token attends to these entries instead of the tokens. Compressed Sparse Attention (CSA) builds one entry per 4 tokens. Like DSA, its indexer scores all earlier entries, and its core attention attends to the 512 entries with the highest scores. Heavily Compressed Attention (HCA) builds one entry per 128 tokens and attends to all earlier entries. DeepSeek-V4-Flash alternates CSA and HCA in 41 of its 43 layers and uses the sliding window alone in the other two. The CSA indexer and HCA compute one pair per 4 and per 128 earlier tokens, about $A/4$ and $A/128$ pairs per request, and their time grows with A. The sliding window, the CSA core attention, and the compression take a fixed time per uncached token, and

their time per request thus grows with t_{uc}. The terms $c_{n,2} \sum A$ and $c_{n,1} \sum t_{\text{uc}}$ in Equation (1) thus capture both parts.

B.3 MoE weights in the decode iteration-time model

In an MoE model, a decode iteration reads the dense weights in full but reads a routed expert only when some token selects it. The dense weights, such as the attention projections and the shared experts, serve every token, and reading them is part of the fixed cost $d_{n,0}$ in Equation (2). In each MoE layer, a gate assigns every token to k of the E routed experts. The layer then reads the weights of each selected expert from GPU memory once and applies them to all tokens that select this expert. It does not read the weights of experts that no token selects. Our engines split every expert across the GPUs of a node with tensor parallelism, and each GPU reads its share of every selected expert. The time to read the routed experts thus grows in proportion to the number of selected experts. Equation (2) writes this time as $d_{n,1}\phi(B)$, where $d_{n,1}$ is the time to read all routed experts and $\phi(B)$ is the fraction that the iteration selects.

We derive $\phi(B)$ under two assumptions. First, every expert receives the same share of tokens. MoE models are trained to keep the load of their experts balanced [12], and each token thus selects a given expert with probability k/E. Second, different tokens select their experts independently. Each of the B running requests feeds t_{fwd} tokens into every MoE layer, and the layer thus routes $t_{\text{fwd}}B$ tokens. The probability that none of them selects a given expert is $(1 - k/E)^{t_{\text{fwd}}B}$. The expected fraction of selected experts is one minus this probability, which gives $\phi(B) = 1 - (1 - k/E)^{t_{\text{fwd}}B}$ in Section 5.1.

B.4 Context length in the decode iteration-time model

The decode iteration-time model omits the KV that an iteration reads for its running requests. This KV grows with their total context but stays small next to the weights that the iteration reads, for two reasons. First, recent models store little KV per token. With MLA, a token of GLM-5.2 takes about 50 KB of KV in FP8, about 50× less than under full multi-head attention (Appendix A.5). Second, sparse attention lets decode read only part of this KV. GLM-5.2 reads the full KV of at most 2048 selected tokens per layer and only the small indexer keys of all other tokens. DeepSeek-V4-Flash reads the full KV of its 128-token window and 512 selected entries, and of all other tokens only compressed entries or indexer keys that each cover 4 or 128 tokens (Appendix B.2). As a result, one million more context tokens lengthen a decode iteration by about 1 ms for GLM-5.2 on GB300 and by at most 1.5 ms for DeepSeek-V4-Flash on H200, whereas an iteration with 8 to 64 running requests takes 17 to 45 ms (Figure 5d and Figure B.1d).

C Extended Analysis of Stationary Workloads and the Shifting Workload Composition

This appendix explains the gains of Evolving Infra on the stationary workloads of Section 6.2 and under the shifting workload composition of Section 6.3. For the stationary workloads, Appendix C.1 compares the phase switches and KV reuse of Evolving Infra and Dynamo, Appendix C.2 shows how the PD split of each system changes over time, Appendix C.3 compares how evenly they balance prefill load, and Appendix C.4 relates their throughput to their E2E latency. For the shifting workload composition, Appendix C.4 also compares the throughput of Evolving Infra, Dynamo, and the best static split in each period. The Dynamo baseline in Section 6 and Appendices C and D corresponds to release v1.5.1 [40].

C.1 Phase Switches and KV Reuse

Table C.1 lists, for every setting of Figure 6, the phase switches of each system in the measurement period, the mean time per switch, and the lost KV reuse. For Dynamo, the switch time runs from the planner decision to the point at which the relaunched engine is ready. For Evolving Infra, it runs from the planner decision to the point at which the node serves requests of its new phase, including the time to drain the requests running on the node. A phase switch of Evolving Infra takes **6.2** s on average, over **100**× shorter than the **754** s that Dynamo needs to relaunch an engine, and retains the KV of the switching node. Evolving

Infra thus loses **81–95%** less KV reuse than Dynamo in every setting, even on Putnam, where it switches more often. As a result, the fraction of prompt tokens that Dynamo prefills without cached KV, one minus its KV hit rate in Table C.1, is 1.29–2.99× that of Evolving Infra.

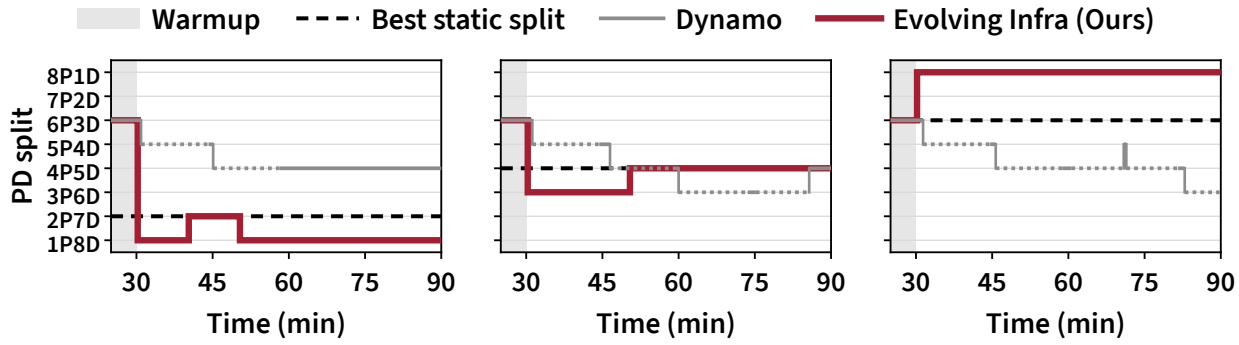
On CC, the KV reuse that Dynamo loses has two main causes (Section 3.3). We attribute a loss to eviction when the request runs on the same prefill node as the previous request in its branch, and to routing when it runs on a different one. Eviction accounts for 39%, 34%, and 46% of the lost KV reuse with 90, 108, and 126 agents, because LRU eviction removes the KV of branches that pause for tools, sub-agents, or users. Routing accounts for another 22%, 17%, and 15%.

Table C.1: **Phase switches and lost KV reuse on stationary workloads.**

		Dynamo				Evolving Infra			
Workload	Concurrent agents	Switches	Mean switch time (s)	KV hit rate (%)	Lost KV reuse (%)	Switches	Mean switch time (s)	KV hit rate (%)	Lost KV reuse (%)
Putnam	224	2	798	96.59	2.47	3	2.3	98.54	0.48
	256	2	798	96.56	2.48	2	3.5	98.85	0.17
	288	3	800	96.42	2.63	5	3.4	98.56	0.46
CC	90	4	681	92.25	5.13	1	0.7	97.08	0.32
	108	4	831	92.62	4.89	2	1.9	96.87	0.48
	126	4	813	89.52	7.78	2	10.8	96.25	1.01
OSWorld	128	5	666	89.79	4.00	1	19.4	93.79	0.21
	144	4	730	92.01	1.85	1	18.0	93.79	0.19
	160	4	748	90.67	3.24	1	17.7	93.84	0.17

C.2 PD Split over Time

Figure C.1 shows the PD split of both systems over the run for one setting of each agent workload. Evolving Infra changes its split at the first planner decision after warmup. It moves OSWorld to 8P1D and holds this split, moves CC to 3P6D and later to 4P5D, and alternates between 1P8D and 2P7D on Putnam. On OSWorld and CC, 8P1D and 3P6D are the best static splits on the symmetric engines of Evolving Infra (Appendix D.1). Dynamo changes its split by one node per relaunch. Each relaunch takes **10–15** minutes, during which only eight nodes serve requests, and these relaunches cover **44–91%** of the measurement period, depending on the setting. Across all nine settings of Figure 6, Dynamo deviates from the best static split of each workload under SGLang PD disaggregation for **41–100%** of the measurement period.



(a) Putnam, 224 agents (b) CC, 126 agents (c) OSWorld, 160 agents

Figure C.1: **PD split of Evolving Infra and Dynamo over time.** The Dynamo line is dotted while one of its nodes relaunches and only eight nodes serve requests.

C.3 Prefill Load Balance

Table C.2 reports the mean time that a prefill request waits in the queue of a prefill node and, as a measure of load balance, the queueing time on the most loaded prefill node relative to the average across prefill nodes, computed per minute and averaged over the measurement period. On CC, Evolving Infra keeps this ratio at **1.75–1.85**×, compared with **2.29–2.61**× under Dynamo, and reduces the queueing time by **2.6–7.6**×.

Table C.2: **Prefill queueing time and load balance on CC.**

	Dynamo		Evolving Infra	
Concurrent agents	Queueing time (s)	Most loaded / average	Queueing time (s)	Most loaded / average
90	11.4	2.29	1.5	1.79
108	11.0	2.38	2.2	1.75
126	11.2	2.61	4.3	1.85

C.4 Throughput and End-to-End Latency

The stationary runs serve a fixed number of agents in a closed loop (Section 6.1). A branch issues its next request only after its previous request completes and the pause between the two requests ends, during which the branch waits for tools, sub-agents, or users. By Little's law [29], the throughput X of a system satisfies

$$X = \frac{\bar{N}_{\text{branch}}}{\bar{T}_{\text{E2E}} + \bar{T}_{\text{pause}}}, \tag{C.1}$$

where $\bar{N}_{\text{branch}}$ is the mean number of requests in flight plus the mean number of branches in a pause, $\bar{T}_{\text{E2E}}$ is the mean E2E latency, and $\bar{T}_{\text{pause}}$ is the total pause time divided by the number of requests. In each setting, the two systems keep a similar $\bar{N}_{\text{branch}}$, for example 159 and 161 on CC with 126 concurrent agents. The lower $\bar{T}_{\text{E2E}}$ of Evolving Infra therefore translates into a higher throughput (Table C.3).

Table C.3: **Throughput, mean E2E latency, and pause between requests on stationary workloads.**

		Dynamo			Evolving Infra		
Workload	Concurrent agents	X (req/s)	$\bar{T}_{\text{E2E}}$ (s)	$\bar{T}_{\text{pause}}$ (s)	X (req/s)	$\bar{T}_{\text{E2E}}$ (s)	$\bar{T}_{\text{pause}}$ (s)
Putnam	224	7.72	19.1	10.1	8.81	15.5	10.0
	256	8.37	20.4	10.1	9.20	17.8	10.0
	288	9.12	21.7	10.0	10.15	18.6	9.9
CC	90	2.48	32.2	15.4	5.40	7.9	12.9
	108	2.48	37.7	17.1	5.78	9.9	13.5
	126	2.14	54.4	18.6	5.85	13.2	14.1
OSWorld	128	1.93	57.4	1.0	4.30	24.5	1.4
	144	1.98	66.3	1.4	4.33	28.1	1.4
	160	1.68	84.4	1.2	4.38	31.4	1.3

Within each period, the run with a shifting workload composition (Section 6.3) also serves a fixed number of agents in a closed loop, so Equation (C.1) holds for this run as well. Over the whole run, Evolving Infra reduces the mean E2E latency by 2.15× and 1.33× relative to Dynamo and the best static split, respectively (Figure 7b), and therefore achieves 2.14× the throughput of Dynamo and 1.28× that of the best static split (Table C.4). Its throughput is also the highest in every period.

Table C.4: **Throughput (req/s) under the shifting workload composition.** The number in parentheses is the number of concurrent agents, 144 Putnam and 72 OSWorld agents in the mixed period. 5P4D is the best static split.

Period	Dynamo	5P4D	Evolving Infra
Putnam (288)	1.57	3.02	**4.05**
Mixed (144 + 72)	2.21	3.64	**3.99**
OSWorld (144)	2.05	3.10	**4.43**
Whole run	1.94	3.25	**4.16**

D Additional Experiments

Beyond the settings of Section 6, we evaluate Evolving Infra under larger and more diverse deployments. Evolving Infra keeps its gain on 256 GB300 GPUs and, in simulation, on 3,072 GB300 GPUs (Appendix D.2), sustains twice the agent arrival rate of static 4P5D under open-loop arrivals (Appendix D.3), and remains effective when prefill and decode use different parallelism (Appendix D.4). Moreover, Evolving Infra outperforms ThunderAgent, an agent-aware router, since KV capacity does not limit serving on GB300 nodes (Appendix D.5), and its asynchronous KV fetching achieves 1.12× the throughput of upstream SGLang (Appendix D.6). Appendix D.1 explains why its best static split differs from that under SGLang PD disaggregation.

D.1 Best Static Split on the Symmetric Engines

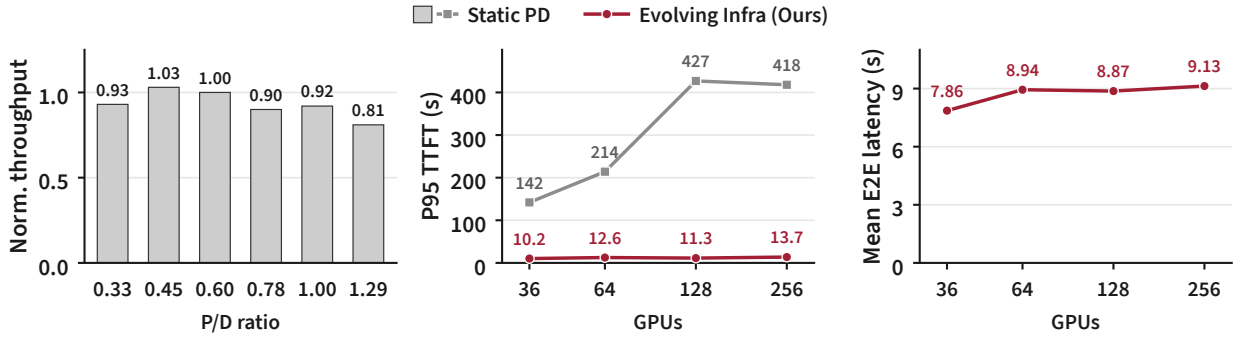
The best static split of Section 6.1 is profiled offline under SGLang PD disaggregation (Figure 2). Evolving Infra serves requests on its symmetric engines (Section 4.1), so its best static split may differ. This subsection compares the best static split of Evolving Infra with that of SGLang on the stationary workloads of Section 6.2.

Experimental Setup. We serve Putnam, CC, and OSWorld with 224, 90, and 128 concurrent agents, the lowest concurrency level of each agent workload in Section 6.2. As in Section 6.1, we serve GLM-5.2 for Putnam and CC and Kimi K2.6 for OSWorld on 36 GB300 GPUs. *SGLang* runs in its PD disaggregation mode at its best static split in Figure 2a, with the router and HiCache settings of our baselines (Section 6.1). *Evolving Infra* runs at its best static split, with online phase-split adaptation disabled and its PD split fixed for the whole run. We score minutes 30 to 85 of each run, and each configuration runs once.

Table D.1: **The best static split differs between SGLang and the symmetric engines of Evolving Infra.** Each row is one run of a system at its best static split, the split with the highest throughput, on 36 GB300 GPUs, scored from minute 30 to minute 85. The number in parentheses is the number of concurrent agents.

		Best static	Throughput	TTFT (s)		E2E latency (s)	
Workload	System	split	(requests/s)	Mean	P95	Mean	P95
Putnam (224)	SGLang	2P7D	9.00	1.0	2.5	14.6	74.0
	Evolving Infra	2P7D	**9.20**	**0.6**	**1.2**	**14.0**	**72.6**
CC (90)	SGLang	4P5D	4.97	4.8	19.5	9.2	30.7
	Evolving Infra	3P6D	**5.44**	**3.5**	**9.7**	**7.8**	**22.2**
OSWorld (128)	SGLang	6P3D	3.12	35.3	50.8	35.8	51.3
	Evolving Infra	8P1D	**4.33**	**21.6**	**35.6**	**24.2**	**39.2**

Results. The symmetric engines shift the best static split from 4P5D to 3P6D for CC and from 6P3D to 8P1D for OSWorld (Table D.1). For Putnam, 2P7D is the best static split on both systems. The shift



(a) Throughput across P/D ratios (b) Tail latency at scale (c) Mean latency at scale

Figure D.1: **Evolving Infra keeps its latency low as the cluster grows to 256 GPUs.** Static PD is SGLang in its PD disaggregation mode with a fixed split. (a) On 256 GPUs, throughput is normalized to that at a P/D ratio of 0.6 (24P40D). (b) From 36 to 256 GPUs, the P95 TTFT of static PD rises about threefold, whereas that of Evolving Infra stays within 10–14 s. (c) The mean E2E latency of Evolving Infra rises by only 16%.

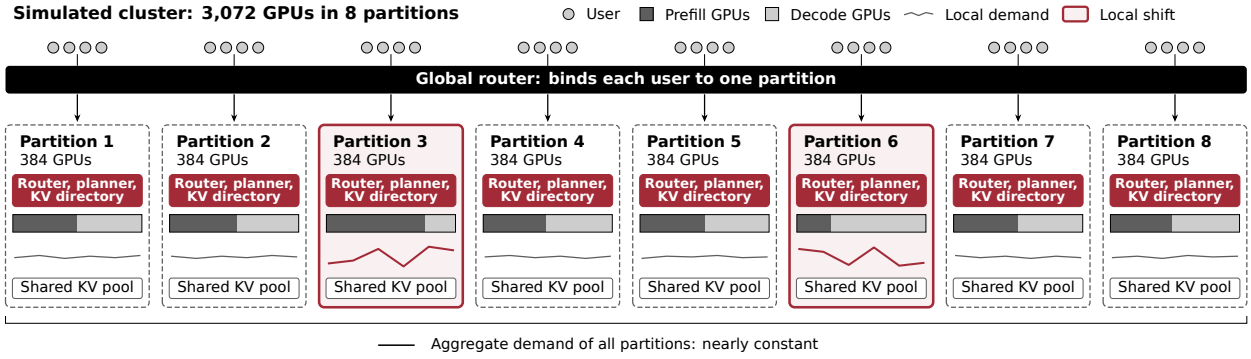


Figure D.2: **Hierarchical serving binds each user to one partition.** In a simulated cluster of 3072 GPUs, a global router binds each user to one of eight partitions of 384 GPUs. Each partition runs its own router, planner, and KV directory. The aggregate demand stays nearly constant, but a few partitions (red) see a local shift in demand, and their planners adapt their PD splits.

is largest for OSWorld and stems from how the two systems allocate the GPU memory of decode nodes. SGLang reserves GPU memory on a decode node for the KV of each request before its prefill begins. It holds this reservation while the request waits in the queue of a prefill node. At 6P3D, these reservations occupy 91% of the KV cache of its three decode nodes on average, so removing a decode node would delay prefill. Evolving Infra instead selects the decode node of a request only when its prefill begins, so queued requests occupy no memory on decode nodes. With a single decode node, the mean E2E latency of OSWorld exceeds its mean TTFT by only 2.6 s (24.2 s versus 21.6 s). Evolving Infra therefore assigns the remaining eight nodes to prefill. With online phase-split adaptation, Evolving Infra reaches 8.81, 5.40, and 4.30 requests/s in these settings (Table C.3), 0.98–1.38× the throughput of the best static split under SGLang.

D.2 Scaling to Larger Clusters

This subsection evaluates Evolving Infra on real clusters of up to 256 GB300 GPUs and on a simulated cluster of 3,072 GPUs. Both show that the aggregate PD demand varies less as the cluster grows, whereas the imbalance within the cluster grows.

Setup on up to 256 GPUs. We serve the Claude Code traces with GLM-5.2 on 36, 64, 128, and 256 GB300 GPUs and scale the number of concurrent agents with the GPUs, 10 agents per GPU. Static PD runs SGLang in its PD disaggregation mode at a fixed split, with the router and HiCache settings of Section 6.1.

The aggregate split matters less at scale. On 256 GPUs, static PD keeps 81–103% of the throughput of 24P40D across P/D ratios from 0.33 to 1.29 (Figure D.1a). The demand of many agents together varies less over time, and each node holds a smaller fraction of the capacity. A mismatch between the split and the aggregate demand thus costs less throughput than on 36 GPUs.

The imbalance among nodes grows. The P95 TTFT of static PD, however, rises about threefold from 36 to 256 GPUs, from 142 s to 418 s (Figure D.1b). With more prefill nodes, the cache-aware router concentrates the requests of more agents on the few nodes that hold their prefixes. The router and the shared KV pool of Evolving Infra spread this load across nodes. Its P95 TTFT thus stays within 10–14 s, and its mean E2E latency rises by only 16% (Figure D.1c).

Offline profiling grows costly at scale. Finding the best static split offline also becomes expensive as the cluster grows. A cluster of N nodes with g GPUs each has $N-1$ candidate splits, and evaluating one candidate replays the whole trace of length T on all Ng GPUs, together with a warmup of length T_{w}. An exhaustive search thus costs $(N-1)(T+T_{\mathrm{w}})Ng$ GPU-hours, or $O(N^2gT)$. A binary search compares two adjacent splits at each of its $\lceil \log_2(N-1) \rceil$ steps and costs $O(NgT \log N)$. For the three-hour CC trace on 256 GPUs, $N = 64$ and $T_{\mathrm{w}} = 0.5$ h, so each candidate costs 896 GPU-hours. The binary search takes 12 runs, or 10,752 GPU-hours, which is 14× the 768 GPU-hours of serving the trace once. The exhaustive search takes 56,448 GPU-hours. Three factors raise these costs further. First, a binary search assumes that throughput first rises and then falls with the number of prefill nodes, yet the throughput in Figure D.1a rises again from a P/D ratio of 0.78 to 1.00. A binary search can thus stop at a local optimum, and a reliable search falls back to the exhaustive one. Second, adjacent splits differ by only a few percent, for example 3% between P/D ratios of 0.45 and 0.60, which is comparable to the variation between runs. Each candidate then needs repeated runs, which multiplies the cost. Third, the search must recur for every new workload composition. A best split per planning window costs even more, since the searches of different windows visit different splits and together approach all $N-1$ candidates. Evolving Infra instead evaluates the $N-1$ candidates with its iteration-time models on m slices at the end of each window, which takes $O(Nm)$ arithmetic on a CPU and no extra GPU time.

Simulation setup. We further simulate a cluster of 3,072 GB300 GPUs with Frontier [16], a discrete-event simulator for LLM serving. We profile the prefill and decode iterations of GLM-5.2 offline on our GB300 nodes and supply these profiles to the simulator. Frontier models KV caching only within GPU memory, and we extend it with the shared host-memory KV pool and the asynchronous fetches of Section 4.2. Since Evolving Infra fetches remote KV while a request waits in the queue, the simulator assumes that each fetch completes before the prefill of its request begins and charges the fetch no time. We replay the Claude Code and Codex traces over 24 hours with about 10 concurrent agents per GPU, or 30,720 agents in total.

Hierarchical scheduling. A single router cannot monitor every node of a cluster with thousands of GPUs. We therefore divide the cluster into 8 partitions of 384 GPUs, each with its own router, planner, and KV directory (Figure D.2). Constraints such as region and hardware bind each user to one partition, and each partition thus serves its own subset of agents.

Local shifts under a stable aggregate. Across all partitions, the aggregate workload stays nearly constant, and the PD ratio of the cluster varies by only 7% over the trace. A static split, however, leaves 2 of the 8 partitions severely imbalanced, with queueing times up to 13.9× the cluster average. These imbalances rarely stem from bursts of load. While a partition is imbalanced, its total work changes by less than 12.3%, whereas its PD ratio shifts by up to 42.4%. The planner of each partition follows these shifts and improves the throughput by 1.42× over a static split.

Summary. At scale, Evolving Infra corrects the imbalance at two levels. Its router and shared KV pool balance the load among the nodes of a cluster, and the planner of each partition follows the local shifts in PD demand, even when the aggregate workload of the cluster is fixed. Neither level requires offline profiling.

D.3 Open-Loop Replay at Fixed Agent Arrival Rates

The stationary runs of Section 6.2 replay a fixed number of agents in a closed loop, so a slower system also receives requests more slowly, and its queues stay bounded. In production, new agents arrive whether or not earlier ones have finished. This subsection lets agents arrive at a fixed rate and measures the highest rate that each system sustains within latency targets.

Experimental Setup. We serve the Claude Code trace with GLM-5.2 on 36 GB300 GPUs with the models and hardware of Section 6.1. New agents arrive in a Poisson process at 0.5 to 1.5 agents per minute in steps of 0.25, and all systems replay the same arrivals at each rate. Within an agent, requests follow the trace as in the stationary runs: each is sent once the request it depends on completes and the recorded delay elapses. At most 258 agents run at once, and the time that an agent waits for admission counts toward the TTFT and E2E latency of its first request. A system sustains a rate if its P50 and P95 TTFT stay within 5 s and 15 s, its P50 E2E latency stays within 10 s, it admits at least 90% of the arriving agents, and fewer than 0.5% of its requests fail. We compare Evolving Infra, which starts from 4P5D, with two baselines of Section 6.1: SGLang PD disaggregation at 4P5D, the best closed-loop static split for CC (Figure 3b), and Dynamo, which starts from 6P3D. In the closed-loop runs of Section 6.4, the initial split changes the mean E2E latency of Evolving Infra by at most 1.8%. We score minutes 30 to 90 of each run, and each configuration runs once.

Table D.2: **Evolving Infra sustains twice the agent arrival rate of static 4P5D.** Each system serves CC once at each arrival rate, and latencies are in seconds. Bold marks the runs that meet every target.

	Evolving Infra			Static 4P5D			Dynamo		
Arrival rate	TTFT		E2E	TTFT		E2E	TTFT		E2E
(agents/min)	P50	P95	P50	P50	P95	P50	P50	P95	P50
0.5	**0.7**	**6.8**	**2.5**	**0.7**	**10.8**	**2.7**	1.0	22.0	3.4
0.75	1.3	17.8	4.4	3.8	57.0	7.3	21.0	234.5	25.7
1.0	**1.2**	**12.3**	**4.6**	6.6	139.0	11.3	11.9	122.1	17.4
1.25	16.8	55.0	21.8	18.5	1,142.7	21.1	16.2	186.9	21.0
1.5	68.4	151.9	74.7	637.8	1,377.1	639.9	19.6	251.1	24.4

Results. Evolving Infra sustains 1.0 agents per minute, twice the rate of static 4P5D, whereas Dynamo misses the targets at every rate, with a P95 TTFT of 22.0 s already at 0.5 agents per minute (Table D.2). At 1.0 agents per minute, Evolving Infra keeps the P95 TTFT at 12.3 s, 11.3× and 9.9× lower than that of static 4P5D and Dynamo, respectively. As the arrival rate grows, its planner gives prefill more nodes: 2–3 at 0.5 agents per minute, 3–4 at 1.0, 5 at 1.25, and 6–7 at 1.5. Evolving Infra misses the targets at 0.75 agents per minute, although it meets them at 1.0, because of a single planner switch: at minute 75, the planner moves to 2P7D, and prefill requests queue for 120–275 s at P95 until it adds prefill nodes back at minute 85. Before this switch, the run meets every target, with a P95 TTFT of 14.0 s instead of 17.8 s.

D.4 Compatibility with Heterogeneous Parallelism

The best parallelism can differ between prefill and decode. For example, Huawei's xDeepServe serves DeepSeek with tensor parallelism (TP) in prefill and attention data parallelism (ADP) in decode, because the two phases see different sequence lengths and ADP avoids duplicating the MLA KV cache [71]. SGLang and vLLM, however, choose parallelism strategy with launch arguments [56, 65], so a node that switches between them must relaunch its engine. The symmetric engine of Evolving Infra can avoid this relaunch, because where a node places its weights can be decoupled from how it computes with them: a node can hold the union of the weights that both phases need and switch phases by changing only its computation path, without moving

any weights. The extra weights take a small share of GPU memory, and the KV cache of Evolving Infra lies mostly in abundant host memory, which holds four times as much KV as GPU memory (Section 6.1).

Experimental Setup. We serve GLM-5.2 [18] on six 8×B200 nodes, with TP8 on prefill nodes and ADP8 on decode nodes, and run BrowseComp agents [67], which can only spawn sub-agents, for two hours. We compare Evolving Infra with Dynamo [40] and the best static split, and Evolving Infra and Dynamo both start from 4P2D. In this deployment, the two phases differ only in the attention layers, and the MoE layers use the same parallelism in both. Under ADP, every GPU holds all attention weights, so it also holds the slice that TP assigns to it. Each node of Evolving Infra therefore keeps the ADP layout of the attention weights in both phases: it computes attention with TP in prefill and with ADP in decode, and a switch changes only which of the two it runs. With FP8 attention weights, this layout takes about 10 GB more per GPU than TP alone, or 6% of the memory of a B200 GPU. The planner needs no change, because it fits the iteration-time coefficients of each node online, and these coefficients already depend on the parallelism of the node (Section 5.1).

Results. As Table D.3 shows, Evolving Infra moves to 2P4D, the best static split, and reaches a mean E2E latency of 15.9 s, within 1.3% of the 15.7 s of that split, which requires profiling each split offline. Switching engines takes Evolving Infra 27 s in total over the two hours. Dynamo ends at 3P3D with a mean E2E latency of 21.9 s, 38% higher than that of Evolving Infra. It spends 913 s, 34× as long, switching engines, because it relaunches the worker of a node at every phase switch (Section 3.1).

Table D.3: **Phase switches with TP8 prefill and ADP8 decode.** Dynamo and Evolving Infra start from 4P2D, and the best static split stays fixed. Engine-switch time sums the time of all phase switches over the two hours.

	Best static	Dynamo	Evolving Infra
Final split	2P4D	3P3D	2P4D
Mean E2E latency (s)	15.7	21.9	15.9
Engine-switch time (s)	0	913	27

Wide expert parallelism. The same decoupling carries over to wide EP, where one engine spans an EP group of several nodes and attention runs with ADP. Under wide EP, the two phases use groups of different sizes and different communication kernels. SGLang, for example, evaluates prefill on 4-node groups with kernels built for throughput, and decode on 9-node groups with low-latency kernels captured in CUDA graphs [61]. A phase switch therefore moves a node from a group of one phase to a group of the other. Elastic EP performs such a move without a restart [3], but it transfers weights: when vLLM grows a DeepSeek-V4-Pro group from 16 to 20 GB200 GPUs, the scale-up takes 92 s in total to transfer weights, warm up, and capture CUDA graphs [2]. Evolving Infra avoids most of this weight transfer by again holding the union of what both groups need. Every node already holds the non-expert weights. For the experts, each group keeps two copies of every expert on different nodes, so each GPU holds twice its share. A node can then leave a group without taking away the last copy of any expert, and it can join any group at least half as large as its old one by serving experts that it already holds. For GLM-5.2 with NVFP4 experts, the second copy takes 7% of B200 memory in a 4-node group and 3% in a 9-node group. Nor does a move reload kernels, because one engine can load the kernels of both phases, as the auto mode of SGLang does [61].

D.5 Comparison with Agent-aware Routers

ThunderAgent [23] pauses agents to prevent KV-cache thrashing (Appendix A.4). Section 3.3 instead attributes the KV reuse that existing systems lose to where they place requests and which KV they evict. This subsection tests which explanation holds on GB300 nodes by comparing ThunderAgent with cache-aware routing and Evolving Infra.

Table D.4: **KV capacity does not limit serving on GB300 nodes.** Each column is one run of the Claude Code trace with 126 concurrent agents on GLM-5.2, 36 GB300 GPUs, and a fixed 4P5D split, scored from minute 30 to minute 85. The ideal KV hit rate counts every prompt token whose KV an earlier request produced. Lost KV reuse is the share of these tokens that a run recomputes.

	Cache-aware	ThunderAgent	Evolving Infra
Throughput (requests/s)	4.66	4.43	**6.27**
TTFT, mean / P95 (s)	15.3 / 60.3	17.5 / 59.8	**4.5** / **14.0**
E2E latency, mean / P95 (s)	19.7 / 66.7	22.0 / 67.1	**11.2** / **32.6**
TPOT, mean (ms)	4.6	4.4	7.9
KV hit rate, actual / ideal (%)	94.9 / 97.2	94.7 / 97.2	96.6 / 97.3
Lost KV reuse (%)	2.4	2.6	**0.7**
Uncached prompt tokens per request	7,660	8,119	**5,268**
Peak KV demand of active agents / capacity	–	0.86	–
Paused requests (%)	–	0	–

Experimental Setup. We serve the Claude Code trace with 126 concurrent agents on 36 GB300 GPUs with GLM-5.2, which uses DSA (Appendix B.2). All three configurations keep the split at 4P5D (Table D.4). The first two run the SGLang engines of our baselines (Section 6.1), which evict host memory in LRU order, and differ only in how they choose the prefill node. Each of their prefill nodes caches up to 2.06 million tokens of KV in GPU memory and 8.25 million tokens in host memory, which holds one copy per node (Appendix A.5). For *cache-aware* routing, we reimplement the policy of the SGLang Model Gateway [58]. For *ThunderAgent*, we port the program scheduler that Dynamo ships as a router plugin [23, 40] and keep its default thresholds. Since the plugin targets aggregated serving, we apply it to the choice of the prefill node and count host memory in the capacity of a node. ThunderAgent pauses agents that wait for tools once the contexts of the active agents on a node exceed 95% of this capacity. The harness marks the last request of each agent, so ThunderAgent releases finished agents at once. The Evolving Infra configuration is the run with a fixed split in Section 6.4. We score minutes 30 to 85 of each run, and each configuration runs once.

Results. ThunderAgent pauses no request (Table D.4): by its own estimate, the active agents on a prefill node need at most 86% of its KV capacity (76% on average), below its 95% threshold, since the deduplicated host tier absorbs thrashing (Appendix A.5). It thus performs like cache-aware routing, with 2.6% versus 2.4% lost KV reuse, 0.95× the throughput, and 1.14× the mean TTFT. With the same split and host-memory ratio, Evolving Infra achieves **1.35**× the throughput and 0.29× the mean TTFT of cache-aware routing and loses only 0.7% of the KV reuse, because its prefill nodes fetch KV from any node that holds it (Section 4.2) and evict first the copies that no active branch references (Section 4.3). Its higher TPOT coincides with larger decode batches, and this comparison does not separate the effects of routing and the data plane. On GB300 nodes, KV reuse therefore depends on where KV resides and which copies host memory keeps, not on KV capacity (Section 3.3).

D.6 Async KV Fetching

When a prefill node fetches KV from host memory, it restores the KV into GPU memory and broadcasts it to all GPUs of the node (Section 4.2). This subsection compares the load-back of upstream SGLang with the asynchronous KV fetching of Evolving Infra.

Experimental Setup. We use the setup of Appendix D.5: the Claude Code trace with 126 concurrent agents, GLM-5.2 on 36 GB300 GPUs, a fixed 4P5D split, and deduplicated host memory (Appendix A.5). The router uses cache-aware routing for prefill nodes and round-robin for decode nodes in both runs, and only the engine data plane differs. *Upstream SGLang* broadcasts the KV to the GPUs of the node with NCCL collectives, which share the GPUs with the prefill computation. Evolving Infra moves the KV between GPUs with their copy engines over NVLink while the request waits in the queue, and its decode nodes write

Table D.5: **Asynchronous KV load-back.** Each column is one run of the Claude Code trace with 126 concurrent agents on GLM-5.2, 36 GB300 GPUs, and a fixed 4P5D split with the same routing, scored from minute 30 to minute 85. Idle GPU time counts the time between iterations while a node has requests to serve.

	Upstream SGLang	Evolving Infra
Throughput (requests/s)	4.62	**5.18**
TTFT, mean / P95 (s)	15.7 / 56.4	**12.1** / **42.8**
E2E latency, mean / P95 (s)	20.2 / 64.5	**16.5** / **50.7**
TPOT, mean / P95 (ms)	4.4 / 8.1	4.5 / 8.2
Idle GPU time, prefill / decode (%)	23.6 / 9.0	**14.4** / **7.8**
KV hit rate, actual / ideal (%)	95.1 / 97.3	94.9 / 97.3
Uncached prompt tokens/s (thousands)	34.3	39.3

KV through to host memory with a GPU kernel that does not block the scheduler. We score minutes 30 to 85 of each run, and each configuration runs once.

Results. With the same routing, the asynchronous load-back of Evolving Infra cuts the idle GPU time of prefill nodes from 23.6% to 14.4% and achieves **1.12**× the throughput of upstream SGLang (Table D.5). Mean and P95 TTFT drop by 23% and 24%, and E2E latency by 18% and 21%, although Evolving Infra serves 15% more uncached prompt tokens per second. TPOT stays within 3%. In upstream SGLang, the fetch delays prefill substantially. A prefill node must load the fetched KV into GPU memory before the layers that read it can run, and the NCCL broadcast of this KV shares the GPUs with the prefill computation. As a result, 93% of its prefill iterations wait more than 10 ms before they start, and waiting for the load-back alone takes up to 8% of the prefill GPU time, about 0.3 s for each iteration that consumes fetched KV. These waits add to the queueing delay of every request that fetches its prefix (Section 3.3).

Fetches on decode nodes. The same runs show why the decode nodes of Evolving Infra only write to the pool (Section 4.2). In upstream SGLang, a decode node also loads reusable KV from its host memory, and the whole batch of the node waits for the load. Decode iterations that start while a load-back is in flight leave the GPUs idle 58.7% of the time, with a mean gap of 82 ms before the forward, about 19 times the mean TPOT of 4.4 ms, against 6.9% for the other decode iterations. Such iterations are rare (0.5% of decode iterations), but each one stalls every request on the node.

Fetches between prefill nodes. A prefill node may receive a request whose prefix KV resides in the host memory of another prefill node. The node then fetches this KV into its own host memory over RDMA (Section 4.2). We profile these fetches in the run without online phase-split adaptation (Figure 8b) from minute 30 to minute 85. As Figure D.3 shows, the fetch time grows almost linearly with the number of fetched tokens. Each token carries 54.6 KB of KV, and the RDMA transfer reaches about 52 GB/s for large fetches. A fetch of 64K–128K tokens therefore takes 129 ms at the median. Most fetches are much smaller. For the 92% of fetches with fewer than 16K tokens, the median fetch time is 7 ms. Evolving Infra starts the fetch as soon as the request arrives at the prefill node that serves it. The fetch therefore overlaps with the *queueing delay* of the request, i.e., the time from this arrival to the start of its prefill. The median queueing delay is 0.7–2.2 s, at least 3.9 times the median fetch time for every fetch size. A fetch that starts only when the prefill begins would instead add its full time to the TTFT.

E Additional Details

This section gives the three algorithms of Evolving Infra in pseudocode. Algorithm E.1 and Algorithm E.2 implement the branch counts of the cluster-wide KV directory (Section 4.3) and the host-memory eviction of Section 5.3. Algorithm E.3 combines the routing of Section 5.3 (inner loop) with the phase-split planner of Section 5.2 (outer loop).

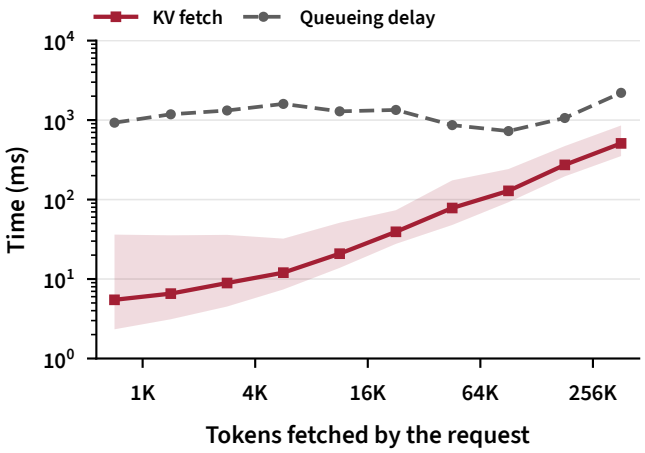


Figure D.3: **Time of KV fetches between prefill nodes.** Same configuration as the run without online phase-split adaptation in Figure 8b. Lines show medians over power-of-two bins of fetched tokens, and the band shows the 10th to 90th percentiles of the fetch time. The fetch time grows almost linearly with the fetched tokens and stays well below the queueing delay, i.e., the time from the arrival of a request at its prefill node to the start of its prefill.

E.1 Branch and version identifiers

A branch is an independently maintained context and its sequence of requests (Section 1). The harness tags each request with the ID β of the branch that issues it. When it forks a context or spawns a sub-agent, it uses a new ID for the new branch, and it reports when a branch finishes. The router keeps, for each branch, its version v_β, the prefix-tree path π_β of its last request, and the nodes N_β where it holds counts. A path is the sequence of KV blocks of a context in the prefix tree of the KV directory. The version tells an appended context from a rewritten one (Algorithm E.1): a request that only appends tokens extends the previous path and keeps the version, whereas a request whose context was rewritten, e.g. by compaction, leaves the previous path and increments the version.

Algorithm E.1 Branch ID and version of a request

Input: request q with branch ID β from the harness; prefix tree $\mathcal{T}$ of the KV directory; branch table S
Output: version v_β, previous path π_{old}, new path π_{new}

$\pi_{\text{new}} \leftarrow$ path of the full pages of the input of q in $\mathcal{T}$, including the blocks that the prefill of q adds
if $\beta \notin S$ **then**
  $S[\beta] \leftarrow (v_\beta{=}1,\ \pi_\beta{=}\emptyset,\ C_{\beta,n}{=}\emptyset$ for every node $n)$ {new, forked or sub-agent branch}
  $\pi_{\text{old}} \leftarrow \emptyset$
else
  $\pi_{\text{old}} \leftarrow \pi_\beta$
  if π_{old} is not a prefix of π_{new} **then**
    $v_\beta \leftarrow v_\beta + 1$ {context rewritten, e.g. by compaction}
  end if
end if
$\pi_\beta \leftarrow \pi_{\text{new}}$
return $(v_\beta, \pi_{\text{old}}, \pi_{\text{new}})$

A forked branch starts with a new ID and an empty previous path, so the node that serves it adds one count to every block that it reuses from its parent. A sub-agent starts a new context, so its blocks are new to the tree. Neither case changes the counts of the parent branch.

E.2 Branch counts and host-memory eviction

The directory keeps, for each block b and each node n that holds it, the count $R_{b,n}$ of active branches that reference b on n. Algorithm E.2 updates these counts from the output of Algorithm E.1 and from KV transfers, and each node evicts its host blocks in the order that the counts give (Section 5.3).

Algorithm E.2 Branch counts in the KV directory and host-KV eviction

Input: counts $R_{b,n}$; branch table S with v_β, π_β, and the counted blocks $C_{\beta,n}$ of each branch on each node; requests, placements, KV transfers, branch ends, and phase switches
Output: updated counts; host blocks to evict on each node

Count(β, n, π): {make β count exactly the blocks of π on n}
  $R_{b,n} \leftarrow R_{b,n} + 1$ for $b \in \pi \setminus C_{\beta,n}$; $R_{b,n} \leftarrow R_{b,n} - 1$ for $b \in C_{\beta,n} \setminus \pi$
  $C_{\beta,n} \leftarrow \pi$; $N_\beta \leftarrow \{m : C_{\beta,m} \neq \emptyset\}$
Release(β, n): Count$(\beta, n, \emptyset)$
on request of branch β with $(v_\beta, \pi_{\text{old}}, \pi_{\text{new}})$ from Algorithm E.1:
if v_β increased **then**
  Count$(\beta, m, C_{\beta,m} \cap \pi_{\text{new}})$ for $m \in N_\beta$ {release the dropped history once per node}
end if
on placing β on node n (prefill routing or decode binding):
  Count(β, n, π_β) {new blocks +1; reused blocks keep their counts}
  for all $m \in N_\beta \setminus \{n\}$ in the phase of n: once n holds version v_β of β, Release(β, m) {P2P or D2D move}
  nodes of the other phase keep their counts {P2D or D2P: both copies serve later steps}
on branch end of β: Release(β, m) for $m \in N_\beta$; remove β from S
on phase switch of n from D to P: Release(β, n) for every β with $n \in N_\beta$ {reset the counts on n}
on phase switch of n from P to D: keep the counts on n
EvictHost(n, s): $\mathcal{H} \leftarrow$ unpinned blocks in the host memory of n
while less than s bytes freed **and** $\mathcal{H} \neq \emptyset$ **do**
  $b^* \leftarrow \arg\min_{b \in \mathcal{H}} \big(\mathbb{1}[R_{b,n} > 0],\ T_b\big)$ {zero counts first, LRU within each group}
  free the host copy of b^*; $\mathcal{H} \leftarrow \mathcal{H} \setminus \{b^*\}$
  remove b^* from $C_{\beta,n}$ for every β; $R_{b^*,n} \leftarrow 0$
end while

In Algorithm E.2, T_b is the last access time of block b and the minimum follows lexicographic order. A block that an active branch still references on n is evicted only after no zero-count block remains, which keeps the KV of a branch that waits for a tool and evicts the history that compaction replaced or the source copy of a moved branch.

E.3 Routing and phase-split planning

Algorithm E.3 runs two loops. The inner loop routes each request to the prefill node with the shortest predicted TTFT, Equation (7). For each prefill node n, the router forms the work of the request on n: the cached prefix $t_{\text{c},n}$ is the longest prefix that the KV directory finds on n or on another node, of which x_n tokens must be fetched from another node, and $t_{\text{uc},n}$ counts the remaining input tokens. The router reads the running batch and the queue of n from the latest state that n reports, and adds the requests that it has sent to n but that n has not reported yet. It sums the service times of Equation (3) to predict the wait $\hat{W}_n$ before the request starts. The fetch runs while the request waits, so only the part of the fetch time $\tau(x_n)$ that exceeds the wait adds to the prediction. The prediction ends when the prefill of the request finishes, so it leaves out the first decode iteration, which is the same on every prefill node. Our implementation evaluates the sum by replaying the chunked prefill iterations of n in arrival order, which also accounts for requests that share an iteration. The decode node is bound when the prefill starts. The outer loop is the planner of Section 5.2.

Parameters of our runs. Each prefill node reports its state at every iteration launch, and the router uses a state only if it is at most 5 s old. The chunk size is 8,192 tokens. $\tau(x) = 0.45\,\text{s} + 1\,\mu\text{s} \cdot x$ for $x \geq 512$

tokens (no fetch below 512 tokens). The router also adds to $\hat{T}_n$ a copy price proportional to x_n, which charges the duplicate KV that a fetch creates. $T_{\text{learn}} = 1{,}800\,\text{s}$, $T_{\text{win}} = 600\,\text{s}$, and 60-s slices, so $m = 10$ for a full window. The planner skips the 120 s after each switch.

Algorithm E.3 Online adaptation: prefill routing (inner loop) and phase-split planning (outer loop)

Input: nodes $\mathcal{N}$, initial split $(\mathcal{P}, \mathcal{D})$, window T_{win} with m slices, warmup T_{learn}, fetch time $\tau(\cdot)$, copy price κ

Serve T_{learn} in a collocated mode; fit the iteration-time models of each node; freeze the coefficients; start PD disaggregation from $(\mathcal{P}, \mathcal{D})$
loop
  for all request r that arrives in the window **do**
    Update the branch of r with Algorithm E.1 and Algorithm E.2
    for all $n \in \mathcal{P}$ **do**
      $t_{\text{c},n} \leftarrow$ longest prefix of r cached on n or on another node; $x_n \leftarrow$ its tokens that only another node holds
      $w_n \leftarrow$ prefill part of the work of r, with $t_{\text{uc},n} = t_{\text{in}}(r) - t_{\text{c},n}$ and $t_{\text{c},n}$; $Q_n \leftarrow$ queue of n plus requests sent to n but not yet reported
      $\hat{W}_n \leftarrow T_n^{\text{rem}} + \sum_{j \in Q_n} T_{\text{P},n}^{\text{req}}(w_j)$ {wait before r starts}
      $\hat{T}_n \leftarrow \hat{W}_n + T_{\text{P},n}^{\text{req}}(w_n) + \max\big(0,\ \tau(x_n) - \hat{W}_n\big) + \kappa\, x_n$ {Eq. (7) plus the exposed fetch and the copy price}
    end for
    $n^* \leftarrow \arg\min_{n \in \mathcal{P}} \hat{T}_n$, ties to the node with the longest prefix of r in its own memory; append r to Q_{n^*}
    Send r to n^*, which fetches the x_{n^*} tokens while r waits; place the branch of r on n^* (Algorithm E.2)
    When n^* starts to prefill r, bind r to the node in $\mathcal{D}$ with KV room for r and the fewest requests, and place the branch of r on that node
  end for
  Divide the window into m slices; measure $\bar{w}_i, \bar{q}_i$ on each slice i; remeasure $\bar{t}_n$ and f_n of each prefill node
  if no node is still draining from the last switch **and** at least 120 s have passed since it **then**
    $\mathcal{C} \leftarrow \emptyset$
    for all prefill count $p \neq |\mathcal{P}|$ **do**
      $(\mathcal{P}_p, \mathcal{D}_p) \leftarrow$ split with p prefill nodes that changes the fewest nodes
      $g_i \leftarrow \Theta(\mathcal{P}_p, \mathcal{D}_p; \bar{w}_i, \bar{q}_i) - \Theta(\mathcal{P}, \mathcal{D}; \bar{w}_i, \bar{q}_i)$ for $i = 1, \ldots, m$
      $\bar{g}_p, \sigma_p \leftarrow$ mean and standard deviation of $g_1, \ldots, g_m$
      if $\bar{g}_p > 2\sigma_p/\sqrt{m}$ **then**
        $\mathcal{C} \leftarrow \mathcal{C} \cup \{p\}$
      end if
    end for
    if $\mathcal{C} \neq \emptyset$ **then**
      $p^* \leftarrow \arg\max_{p \in \mathcal{C}} \bar{g}_p$; apply $(\mathcal{P}_{p^*}, \mathcal{D}_{p^*})$ through the phase switch
    end if
  end if
end loop